

UNIVERSITY OF SALERNO

DEPARTMENT OF INFORMATION AND ELECTRICAL ENGINEERING
AND APPLIED MATHEMATICS



Master's Degree Program in Computer Engineering

T-BOT: The Navigation Robot for Optimized Multi-Agent Exploration

Supervisor:

Prof. Alessia SAGGESE

Candidate:

Beniamino Squitieri

Mat. 0622702021

ACADEMIC YEAR 2023/2024

*Dedicated to those who try every day and never give up in the face of
difficulties.*

Contents

1	Introduction	7
1.1	Problem Definition	7
1.2	Relevance in Computer Engineering	8
1.3	Summary of Contributions	8
1.4	Organization of the Thesis	9
1.5	Final Remarks	10
2	State of the Art	11
2.1	Mapping	11
2.1.1	Motion Model and Sensor Model	13
2.1.2	Occupancy Grid Maps	14
2.1.3	Topological Maps	15
2.2	Localization and Navigation	16
2.2.1	Localization	16
2.2.2	Navigation	17
2.2.3	SLAM Techniques (Simultaneous Localization and Map- ping)	18
2.3	Voronoi Diagrams for Topological Map Generation	18
2.4	Detection of Virtual Doors and Map Optimization	19
2.5	Graph Traversal Algorithms and Applications	21
2.5.1	Overview of Key Algorithms	21
2.6	Applications of Complete Graph Traversal Algorithms	22

CONTENTS

2.7	Multi-Robot Coordination and Fleet Management	24
2.7.1	Task and Robot Constraints	24
2.7.2	ROS2 for Multi-Robot Coordination	25
2.7.3	Strategies for Avoiding Overlaps and Improving Efficiency	26
2.7.4	Robot Coalitions	27
3	Used tools and hardware	28
3.1	Robot Description: TurtleBot 4	28
3.1.1	Hardware Overview	29
3.1.2	Power and Connectivity	29
3.1.3	Software Environment	29
3.1.4	Applications	30
3.2	ROS2 (Robot Operating System 2)	30
3.2.1	Enhanced Real-Time Performance	31
3.2.2	Modular and Flexible Architecture	31
3.2.3	Security Enhancements	32
3.2.4	Sensor Integration and Data Management	33
3.2.5	Key Packages and Tools	33
3.2.6	RViz	34
3.3	Libraries Used	36
3.3.1	OpenCV	37
3.3.2	NetworkX	37
3.3.3	Additional Libraries	39
4	Original Contribution to the Solution of the Problem	40
4.1	High-Level System Operation	41
4.1.1	Pipeline Overview	41
4.1.2	Master-Slave Coordination	43
4.1.3	Slave Robot Execution	43
4.1.4	Overall Workflow	43
4.2	Topological Map Generation from an Occupancy Grid Map . . .	44
4.2.1	Modular Structure of the Codebase	45

CONTENTS

4.3	Map Cleaning and Preparation	46
4.3.1	Morphological Cleaning Pipeline	47
4.3.2	Outcome and Applications	47
4.4	Skeletonization and Voronoi Diagram Techniques	48
4.4.1	Binary Map Generation	48
4.4.2	Distance Map Calculation	48
4.4.3	Skeletonization	49
4.4.4	Implementation Considerations	50
4.5	Algorithmic Implementation Overview	50
4.5.1	Morphological Operations	50
4.5.2	Euclidean Distance Transform (EDT)	50
4.5.3	Skeletonization	52
4.5.4	Topological Graph Creation	54
4.5.5	Coordinate Transformation	54
4.6	Topological Graph Construction Using Collision-Aware Tech- niques	55
4.6.1	Overview	55
4.6.2	Graph Construction	56
4.6.3	Connecting Disconnected Components	56
4.6.4	Graph Representation	56
4.7	Design and Choice of Navigation Algorithm	57
4.7.1	Choice of Algorithm	57
4.7.2	Design and Adaptation of the Algorithm	59
4.8	Introduction to the Chinese Postman Problem (CPP)	60
4.9	Definition and Key Concepts	61
4.9.1	Algorithm Overview	62
4.9.2	Undirected Graph Solution	62
4.9.3	Directed Graph Solution	63
4.9.4	Applications	63
4.9.5	Variants of the CPP	64
4.9.6	Solution focusing on Unirected Graph Solution	64

CONTENTS

4.9.7	Understanding Matching in Graphs	65
4.9.8	Detailed Explanation of ‘min_weight_matching’ and ‘max_weight_matching’	65
4.9.9	Algorithmic Guarantees and Practical Considerations . .	66
4.9.10	Additional Insights on Eulerian Tour	66
4.9.11	Additional Insights on Matching Algorithms	68
4.10	Connecting and Configuring the TurtleBot 4	75
4.10.1	Initial Setup	76
4.10.2	Advanced Networking: Simple Discovery Setup	77
4.10.3	System Updates and Verification	78
4.11	ROS2 Advanced Distributed Architecture	79
4.11.1	Design of the Proposed System	79
4.11.2	Master Re-election Protocol via the Bully Algorithm . .	85
4.11.3	Scalable Deployment with ROS2 Launch System	86
4.11.4	Hybrid Centralized and Distributed Architecture	87
4.11.5	Advantages of the Distributed Approach	87
4.11.6	Enhanced Heartbeat Mechanism and Master Election Protocol	88
4.11.7	Interaction Between Slave Node and Nav2 Stack	88
5	Experimental Validation and Application Aspects	90
5.1	Introduction	90
5.1.1	Experimental Setup	91
5.2	Map Quality Assessment	92
5.2.1	Noise Reduction Efficacy	93
5.2.2	Structural Integrity	93
5.3	Graph Representativeness Evaluation	93
5.3.1	Collision Avoidance Verification	95
5.3.2	Summary	96
5.4	CPP Algorithm Testing	96
5.4.1	Test Script Development	96
5.4.2	Path Verification	97

CONTENTS

5.4.3	Path Optimality Validation	97
5.5	Application Scenarios	99
5.5.1	Hardware Limitations	99
5.5.2	Real-World Testing of the Fleet Navigation System . . .	100
5.5.3	Summary	108
6	Future Work and Enhancements	109
6.1	Adaptive Kernel Sizing for Morphological Operations	109
6.2	Limitations in Hardware Resources	110
6.2.1	Limitations in Simulation	110
6.2.2	Addressing the Limitations	111
6.3	Conclusion	111
	List of Figures	115
	List of Tables	116
6.3.1	Bibliography	117
	Bibliography	117
	Acknowledgements	121

Chapter 1

Introduction

In recent years, advancements in mobile robotics have significantly impacted various sectors, including industrial automation, search and rescue operations, and environmental monitoring. The ability of mobile robots to autonomously navigate and explore complex environments has opened new avenues for enhancing operational efficiency and effectiveness. A critical component of autonomous navigation is the accurate representation and understanding of the environment, typically achieved through sophisticated mapping techniques.

1.1 Problem Definition

This thesis aims to develop a system that transforms Occupancy Grid Maps (OGMs), widely used in mobile robotics for environment representation, into topological maps suitable for navigation and exploration. Additionally, it seeks to analyze how robot fleets can efficiently navigate and divide exploration tasks within a topological graph, optimizing time and resource utilization during missions.

1.2 Relevance in Computer Engineering

Autonomous navigation of robot fleets is pivotal in computer engineering, especially in critical applications like search and rescue missions, infrastructure inspection, and hazardous environment management. Developing algorithms that optimize robot coordination can significantly reduce exploration times, enhance precision, and improve operational safety.

Computer engineering is essential for integrating various technological components to address these challenges, including the development of intelligent algorithms for real-time environment mapping, autonomous navigation, and efficient task allocation among multiple robots. The proposed system leverages advanced concepts such as computer vision, machine learning, autonomous navigation, and precise localization. Furthermore, it utilizes multi-device communication protocols to enable synchronized operations across different robots and sensors, ensuring coordinated and effective mission execution.

1.3 Summary of Contributions

This thesis provides a comprehensive understanding of the design and implementation of a fleet management system for TurtleBot4 robots. The key contributions are:

- **Map Conversion Algorithms:** Implementing algorithms to transform Occupancy Grid Maps into topological maps suitable for navigation and exploration, using skeletonization techniques and Voronoi diagrams tailored for robotics.
- **Chinese Postman Algorithm Implementation:** Adapting this algorithm for mobile robotics to ensure complete environment coverage by a fleet of TurtleBot4 robots, optimizing area coverage and resource utilization.

- **Multi-Robot Collaboration Strategies:** Developing centralized and decentralized task allocation mechanisms using ROS2 and Nav2 with DDS for efficient communication and coordination among robots.
- **Simulation Configuration and Testing:** Conducting extensive unit tests to validate the developed algorithms and strategies, ensuring robust performance and correctness before real-world deployment.
- **Real-World Testing on TurtleBot4 Robots:** Conducting experiments to address hardware and sensor challenges, ensuring the proposed solutions perform effectively in real-world scenarios.

These contributions aim to provide innovative solutions for autonomous multi-robot navigation, extending theoretical algorithms to practical applications and advancing the state of the art in the field.

1.4 Organization of the Thesis

This thesis is organized as follows:

- **Chapter 2: State of the Art** - Analyzes existing methodologies for converting Occupancy Grid Maps into topological maps, skeletonization, Voronoi diagram techniques, multi-robot coordination and navigation approaches.
- **Chapter 3: Tools and Libraries** - Describes the hardware, software, and libraries used in this project, including the TurtleBot4 platform, ROS2 framework, and key libraries such as OpenCV and NetworkX.
- **Chapter 4: Original Contribution to the Solution of the Problem** - Details the development of the proposed system, focusing on transforming Occupancy Grid Maps into topological maps, cleaning operations, skeletonization, and integrating Voronoi diagram techniques.

- **Chapter 5: Testing and Validation** - Presents the experimental setup, performance metrics, and results from simulations and real-world experiments validating the proposed solutions.
- **Chapter 6: Conclusions and Future Work** - Summarizes the research contributions, discusses their implications, and outlines directions for future advancements in autonomous navigation and multi-robot coordination.

1.5 Final Remarks

Integrating advanced mapping techniques with efficient multi-robot coordination strategies holds significant promise for enhancing autonomous robot fleets. By transforming detailed Occupancy Grid Maps into simplified topological representations, this research facilitates more efficient navigation and exploration, contributing to the broader field of computer engineering and its applications in mobile robotics.

Chapter 2

State of the Art

As mobile robots become increasingly integral to various sectors, including warehouse automation, last-mile delivery, healthcare, and logistics, research in this field has achieved significant progress. Advances in autonomous navigation have enabled robots to seamlessly move through dynamic and unpredictable environments, detect and avoid obstacles, and efficiently reach designated locations. At the same time, significant improvements in mapping technologies have allowed robots to construct and adapt real-time maps of their surroundings, enhancing their situational awareness and operational efficiency.

These advancements have greatly expanded the capabilities of mobile robots, allowing them to perform complex tasks in real-world scenarios with greater safety, reliability, and adaptability. Such progress is crucial for addressing the diverse and evolving needs of modern society, enabling mobile robots to play an increasingly active role in supporting and optimizing operations across a wide range of applications.

2.1 Mapping

In the field of mobile robotics, **mapping** represents a fundamental problem for creating accurate models of the environments in which robots must navigate. Generally, mapping involves acquiring a spatial representation of the environment that is useful for navigation and interaction with the external

world. To construct an effective map, the robot must use various sensors that allow it to obtain detailed information about the environment; among the most common devices are cameras, depth sensors, sonar, laser scanners, and infrared sensors.

These sensors, however, are subject to measurement errors and have intrinsic limitations: for instance, light and sound cannot pass through the walls of a building. These limits complicate the mapping process and require accurate navigation and measurement phases to build a precise representation of the environment. The main critical aspects of mapping include:

Nature of Measurement Errors: Sensor measurements are often affected by noise, which accumulates over time, causing significant errors in the resulting map. This is particularly evident when small rotational or translational errors are amplified, resulting in an inaccurate representation of the spaces.

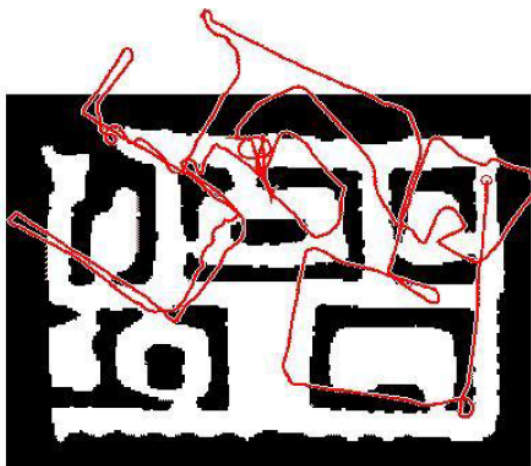


Figure 2.1: Example of accumulated measurement error in the map.

Size of the Mapping Problem: A detailed representation of an environment requires processing a large amount of information, making the mapping problem computationally complex, especially in large environments.

Correspondence Problem: It is necessary to determine whether new sensory measurements are consistent with the map already constructed. This is particularly difficult in environments with few distinctive landmarks.

Dynamic Nature of the Environment: Environments can be dynamic, meaning they are subject to changes over time, such as the closing of a door or the movement of furniture. Such changes affect map consistency and require constant updates.

Exploration: During the map-building phase, the robot often lacks sufficient information about the surrounding environment. Therefore, it is necessary to define exploration strategies that allow the robot to acquire new information and adapt to unforeseen situations.

To address these issues, many mapping algorithms adopt probabilistic approaches, modeling uncertainties and noise through statistical models of the robot's pose and environmental elements. This enables the construction of a robust representation that accounts for measurement uncertainty.

2.1.1 Motion Model and Sensor Model

The existence of various representation problems means that existing mapping algorithms in the literature are often probabilistic, using probabilistic models to describe the robot's pose and elements of the environment. This approach allows for appropriately modeling different types of noise and their effects on sensor data.

The **motion model** of a robot represents the state transition model in mobile robotics. This model is expressed through a conditional probability density:

$$p(x_t \mid u_t, x_{t-1}) \tag{2.1}$$

where x_t and x_{t-1} represent the robot poses, while u_t is a movement command (control) executed by the robot.

The **sensor model** describes the relationship between the robot's position and the sensory observations detected, and is expressed as a conditional probability density:

$$p(z_t \mid x_t, m) \tag{2.2}$$

where x_t represents the robot's pose, z_t is a sensory observation, and m is the map of the environment.

The optimal estimation of the robot's state reduces to a problem of estimating its state x_t given a sequence of sensory observations $(z_{t_0}, z_{t_0+1}, \dots, z_{t_1})$, where $z_t = x_t + n_t$, and n_t represents the noise affecting the measurements. The *likelihood* function is then defined as $p(z_t \mid x)$, the probability of having a particular observation z given the initial state x .

The Maximum Likelihood (ML) criterion states that the estimate closest to the real state is the one that maximizes the likelihood function.

According to the *Maximum A Posteriori (MAP) criterion*, the optimal state estimate is the one that maximizes the function $p(x \mid z)$, calculated based on Bayes' theorem:

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)} = \eta p(z \mid x)p(x) \tag{2.3}$$

where $p(x)$ is the prior distribution and $p(x \mid z)$ is the posterior distribution.

2.1.2 Occupancy Grid Maps

One of the most commonly used methods for mapping static environments is the approach based on **occupancy grids**. In this method, the environment is divided into a regular grid of cells, each representing a portion of space that can be occupied or free. Each cell is associated with a probability that represents the occupancy state of that cell. The *Bayes Filtering* algorithm is often used to update the occupancy probability of cells as the robot gathers new observations through sensors. This approach provides a detailed and continuous representation of the space, particularly suitable for indoor environments like corridors or rooms.

The main advantage of **occupancy grids** is their simplicity of

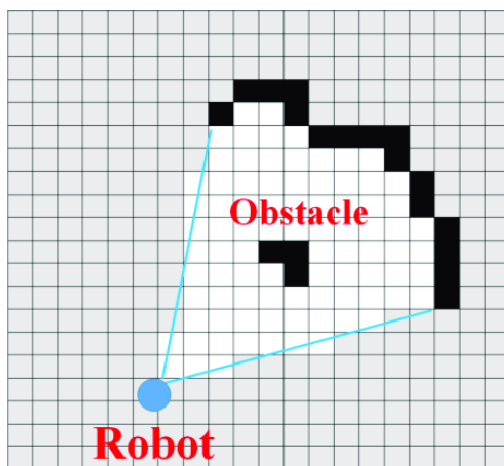


Figure 2.2: Occupancy Grid Map

implementation and ability to provide an accurate representation of the environment. However, the number of cells grows rapidly as the area to be mapped increases, thus increasing computational complexity and memory requirements, especially for high-resolution maps.

2.1.3 Topological Maps

Topological maps represent the environment as a graph of nodes and edges, where each node represents a significant position (e.g., a room or an intersection), and each edge represents a navigable path between two nodes. This type of representation is particularly useful for environments where specific points of interest and paths between them can be identified. Topological maps are more compact compared to occupancy grid maps, as they store only information related to key points in the environment and the connections between them, rather than detailed information about every part of the space.

An important advantage of topological maps is their *computational efficiency*, as they reduce the amount of information to be processed compared to occupancy grids. However, constructing a topological map requires preliminary segmentation of the environment to identify the significant nodes and edges, which can be challenging in environments with few distinctive

features.

2.2 Localization and Navigation

Accurate localization and robust navigation are fundamental requirements for any mobile robot operating in indoor environments. These functionalities enable robots to determine their position and efficiently move within a space while avoiding obstacles and reaching their goals. In our architecture, these capabilities are supported through the integration of established algorithms and tools, ensuring reliability and scalability in real-world applications.

2.2.1 Localization

Localization involves estimating the robot's actual position within the environment based on previous control inputs and sensor data. Several algorithms have been developed for this purpose, each offering specific trade-offs between accuracy and computational complexity. In our system, localization is managed by leveraging the Adaptive Monte Carlo Localization (AMCL) algorithm, a widely used and efficient approach for handling uncertainties in dynamic environments.

Key Algorithms for Localization

- **Markov Localization:** A probabilistic method that evaluates every possible position on the map, updating the probability of each based on sensor observations. While it achieves high accuracy, it is computationally intensive, making it unsuitable for real-time applications in large-scale environments.
- **Kalman Filter Localization:** Starting from a known initial position, the robot refines its estimate continuously using sensor data. The Extended Kalman Filter (EKF) handles non-linear dynamics by

linearizing around the current estimate. This method is faster but requires an accurate initial position and may compromise precision due to linearization.

- **Particle Filtering Localization:** Also known as Monte Carlo Localization (MCL), this method strikes a balance between accuracy and computational efficiency by representing the robot's position using a set of particles, each with an associated probability. Adaptive Monte Carlo Localization (AMCL), the advanced version, dynamically adjusts the number of particles based on the robot's uncertainty, optimizing computational performance.

2.2.2 Navigation

The navigation component of our system is responsible for enabling the robot to move from its current position to a specified goal while avoiding obstacles and adhering to the constraints of the environment. This functionality is built on the Nav2 stack, a powerful framework for autonomous navigation in ROS2.

Path Planning and Execution Nav2 integrates global and local planners to compute collision-free paths and execute them in real-time. The global planner generates an optimal path based on the robot's map, while the local planner dynamically adjusts the trajectory to account for unforeseen obstacles. Together, these planners ensure smooth and efficient navigation.

Dynamic Obstacle Avoidance Dynamic obstacle avoidance is achieved through sensor inputs that continuously update the robot's understanding of its surroundings. This enables real-time reactions to changes in the environment, such as moving objects or sudden obstructions.

2.2.3 SLAM Techniques (Simultaneous Localization and Mapping)

In static environments, the problem of **Simultaneous Localization and Mapping (SLAM)** can be simplified since there are no significant changes over time. The SLAM approach enables the robot to build a map of the environment while localizing itself within it, using sensors such as *LIDAR* or *RGB-D cameras*. In static environments, the main SLAM algorithms, such as *Gmapping* or *Cartographer*, use sensory measurements to iteratively update the map and accurately estimate the robot's position.

A key feature of SLAM in static environments is the stability of the generated map. Since there are no dynamic changes, the map tends to converge quickly to a stable representation, reducing the need for frequent updates. This allows for accurate maps that can be used efficiently for navigation and path planning.

2.3 Voronoi Diagrams for Topological Map Generation

Voronoi diagrams are a powerful tool for deriving topological maps from occupancy grids, as they efficiently represent spatial relationships and structures within an environment. These diagrams divide the space into regions based on proximity, creating a framework for identifying key areas and connections.

Key Steps in Voronoi-based Mapping:

- **Preprocessing:** To ensure accurate mapping, noise and small, irrelevant clusters are removed. The boundaries of the map are clearly defined based on the spatial dimensions relevant to the robot, eliminating unnecessary external data.
- **Simplification of Connections:** The initial Voronoi diagram is refined

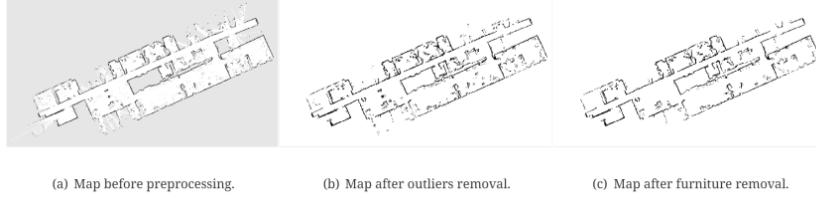


Figure 2.3: preprocessed phase.

to exclude segments that fall outside the defined boundaries or are too close to obstacles. This step ensures a focus on meaningful edges and connection points.

- **Defining Areas:** The environment is divided into regions corresponding to the refined edges of the Voronoi diagram. These regions are iteratively merged and represented as polygons to create a structured view of the environment.

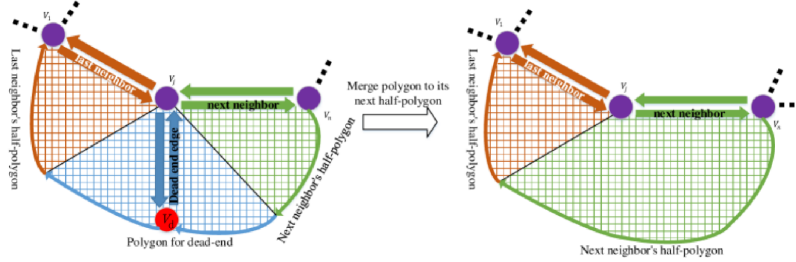


Figure 2.4: Defining Areas

2.4 Detection of Virtual Doors and Map Optimization

Virtual doors serve as proxies for real doors, simplifying the division of spaces in occupancy maps. These are identified by analyzing corner features and angular data in the grid map, followed by segmentation.

Steps for Virtual Door Detection:

- **Distance Mapping:** Calculate the distance of each grid cell to the

2. State of the Art

nearest obstacle, identifying potential door locations based on size thresholds.

- **Feature Validation:** Use radial sampling to confirm the presence of doors, ensuring features meet predefined geometric criteria.
- **Topological Map Formation:** Detected virtual doors form the edges, dividing the environment into nodes (rooms). A genetic algorithm then optimizes the map, merging redundant nodes and reducing edge complexity.

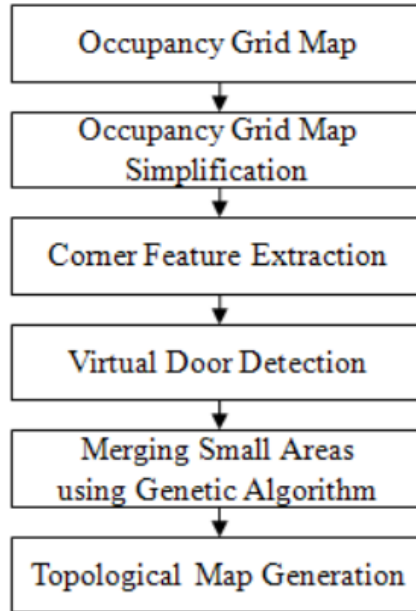


Figure 2.5: Virtual door localization pipeline

This process supports efficient navigation, especially in domestic settings, where tasks like room-specific cleaning benefit from accurate environmental segmentation.

2.5 Graph Traversal Algorithms and Applications

Graph traversal algorithms are fundamental in various applications involving mobile robotics, path planning, network management, and logistics. These algorithms enable the systematic exploration of graph structures, ensuring comprehensive coverage of all edges or nodes while optimizing for efficiency and performance.

2.5.1 Overview of Key Algorithms

Several algorithms have been developed to achieve complete graph traversal, each with its own strengths and suited to different types of graphs and application requirements. Among the most notable are:

- **Chinese Postman Problem (CPP):** An optimization algorithm aimed at finding the shortest possible route that traverses every edge of a graph at least once. Detailed implementation and applications of the CPP are discussed in section 4.8.
- **Fleury's Algorithm:** A straightforward method for finding an Eulerian trail or circuit in a graph, provided certain conditions are met.
- **Hierholzer's Algorithm:** An efficient algorithm for finding Eulerian circuits in graphs where such circuits exist.
- **Modified Depth-First Search (DFS):** Adaptations of the traditional DFS to ensure that every edge is traversed exactly once.
- **Modified Breadth-First Search (BFS):** Variants of BFS tailored for complete edge traversal, useful in specific network exploration scenarios.
- **Backtracking Techniques:** Recursive approaches that explore all possible paths to achieve complete edge coverage, particularly useful in non-Eulerian graphs.

Each of these algorithms offers unique advantages depending on the specific requirements of the application, such as graph size, density, and the presence of dynamic changes. The following sections provide a detailed examination of these algorithms, their implementations, and their respective applications in various domains.

2.6 Applications of Complete Graph Traversal Algorithms

The complete traversal of all edges in a graph has numerous applications across various fields, including logistics, robotics, and network optimization. These algorithms are crucial for ensuring comprehensive edge coverage, which is essential for tasks such as systematic inspections, path optimization, and data acquisition. Below, we highlight some of the key applications relevant to our context:

Infrastructure Maintenance and Inspection

Complete graph traversal is widely employed in infrastructure inspection, such as roads, pipelines, power lines, and other utilities. By ensuring every segment (edge) is traversed exactly once or with minimal redundancy, maintenance crews can systematically inspect critical components, optimizing time and reducing operational costs.

Urban Services and Municipal Planning

Municipal services, such as garbage collection, street sweeping, snow removal, and postal deliveries, benefit significantly from efficient graph traversal. Algorithms ensure optimal coverage of all streets, reducing travel distance and increasing the cost-effectiveness of urban operations.

Robotics and Automated Coverage Path Planning

In robotics, ensuring systematic coverage of an area is vital for tasks such as cleaning, inspection, and mapping. Complete edge traversal algorithms enable autonomous mobile robots to navigate environments comprehensively, ensuring no path is left unexplored. This application is particularly relevant in domains like agriculture and industrial automation.

Logistics and Delivery Optimization

Efficient delivery routing is a critical application of complete graph traversal. By optimizing delivery paths to cover all required locations with minimal travel distance, these algorithms reduce operational costs and improve service efficiency. This is especially relevant for logistics companies aiming to optimize large-scale delivery operations.

Network Design and Communication Optimization

In communication networks, ensuring that all connections are optimally utilized without redundancy is essential for robustness and efficiency. Complete graph traversal plays a vital role in designing cost-effective and reliable network topologies, enabling efficient data flow and robust connectivity.

Traffic and Transportation Systems

Traffic management and transportation systems can leverage graph traversal to optimize road usage and prevent congestion. These algorithms help design efficient routes for vehicles, ensuring that all road segments are utilized effectively while minimizing redundancy and delays.

Geographic and Spatial Mapping

In Geographic Information Systems (GIS), complete graph traversal is used to ensure thorough mapping of roads, trails, or pathways. This enables

efficient data collection and supports navigation tools, aiding in urban planning and resource management.

2.7 Multi-Robot Coordination and Fleet Management

Managing a fleet of mobile robots that navigate simultaneously and distribute the traversal of a graph represents one of the fundamental challenges in robotics and artificial intelligence. This problem is often modeled as **Multi-Robot Task Allocation (MRTA)**, where robots are assigned tasks to optimize specific objectives, considering their capabilities, budget constraints, and environmental restrictions.

Key characteristics of this problem include the heterogeneous nature of tasks, which may require robots with specific abilities, and the need to plan sequences of activities that robots must perform in a particular order. To optimize exploration and task completion, advanced strategies for fleet management, robot communication, and coordination are necessary. This can be achieved using multi-robot coordination techniques, task distribution methods, and solutions like ROS2, which broadly supports multi-robot management.

In a multi-robot coordination context, it is crucial to consider a series of constraints and optimizations to maximize fleet efficiency. Task allocation must account for each robot's specific capabilities, the need to avoid overlaps, and the optimization of exploration to ensure overall effectiveness. Below are some of the solutions and technologies that can be utilized.

2.7.1 Task and Robot Constraints

The problem of task allocation is influenced by several constraints:

Robot Capacity Constraints: Each robot possesses specific capabilities or sensors required to complete certain tasks. Some tasks may

require multiple robots with complementary skills.

Precedence Constraints: Certain tasks must be completed in a specific order, such as opening a door before exploring a room for mapping.

Capacity and Budget Constraints: Robots may have limitations on the number of simultaneous tasks they can handle and their available energy (battery life). Additionally, some tasks might require collaboration, necessitating the formation of robot coalitions.

Temporal and Spatial Constraints: Tasks may have deadlines or need to be executed in specific locations, adding complexity to the task allocation process.

2.7.2 ROS2 for Multi-Robot Coordination

ROS2 Communication and Quality of Service (QoS) ROS2 leverages the **Data Distribution Service (DDS)** middleware, which enhances multi-robot communication through:

- **Distributed Communication:** Each TurtleBot operates as a separate ROS2 node, enabling dynamic discovery and communication.
- **Quality of Service (QoS):** ROS2 allows configuration of communication parameters such as reliability, latency, and bandwidth, ensuring robust data exchange even in dynamic or constrained environments.
- **Action Servers:** A central coordinator node can manage tasks by sending goals to individual robots via ROS2 action servers. This facilitates dynamic task allocation and reallocation based on real-time conditions.

2.7.3 Strategies for Avoiding Overlaps and Improving Efficiency

When multiple robots are involved, it is important to minimize conflicts and overlaps in work areas. Effective strategies include:

Graph Subdivision and Partitioning Utilize graph partitioning algorithms like **METIS**, **Kernighan–Lin**, or **Spectral Partitioning** to divide the environment’s graph into well-connected subgraphs. Each subgraph is then assigned to a different robot, ensuring balanced coverage and minimizing overlap. Advanced partitioning ensures connectivity and balance across subdivisions, reducing the need for inter-robot communication during exploration.

Ant-Colony Optimization (ACO) Inspired by the foraging behavior of ants, **Ant-Colony Optimization (ACO)** can guide multi-robot exploration by using virtual pheromones:

- **Virtual Pheromones:** Robots leave virtual pheromone trails on paths they have traversed, discouraging other robots from repeating the same paths.
- **Dynamic Adaptation:** As robots explore and deposit pheromones, the collective behavior adapts to favor unexplored areas, enhancing overall exploration efficiency.
- **Metaheuristic Nature:** Similar to other metaheuristic algorithms like **Simulated Annealing** and **Genetic Algorithms**, ACO provides approximate solutions that are computationally feasible for large and dynamic environments.

Dynamic Graph Updates Robots continuously share updates about their environment, such as moving obstacles or newly discovered paths. The fleet coordinator uses these updates to adjust task assignments and exploration

strategies through **partial reprogramming**. This adaptability ensures that the fleet remains efficient and responsive to environmental changes without requiring large-scale recalculations.

2.7.4 Robot Coalitions

Some tasks require multiple robots working together, necessitating the formation of **robot coalitions**:

- **Task Requirements:** Identify tasks that cannot be completed by a single robot, such as lifting heavy objects or conducting complex inspections.
- **Coalition Formation:** Select robots with complementary capabilities and align their schedules to collaborate on the task.
- **Coordination Mechanism:** Use communication protocols to synchronize actions and share task-specific information among coalition members.

Chapter 3

Used tools and hardware

In this chapter, the main software tools used for the development and implementation of the robotic navigation system are described. Among the key tools used are **ROS2** and **RViz**, which play a central role in mobile robotics management and environment visualization.

3.1 Robot Description: TurtleBot 4



Figure 3.1: TurtleBot 4 in operation.

The TurtleBot 4 is an advanced, modular robotic platform designed for

3. Used tools and hardware

research, education, and industrial applications. Built on the reliable iRobot Create 3 base, it integrates high-performance computing, an extensive sensor suite, and modular expandability, enabling breakthroughs in autonomous navigation and multi-robot systems.

3.1.1 Hardware Overview

Mobile Base: The iRobot Create 3 base features a differential drive system for agile navigation, integrated IMU and encoders for precise motion tracking, and obstacle detection sensors. It supports a payload of up to 9 kg and is powered by a long-lasting lithium-ion battery.

Computing Unit: The Raspberry Pi 4B (4 GB RAM) serves as the processing hub, running Ubuntu 20.04 with ROS 2 for advanced robotic functionalities. It offers multiple connectivity options and expandable storage for seamless operation.

Sensor Suite: The TurtleBot 4 includes an OAK-D Pro stereo AI camera for advanced vision tasks, an RPLIDAR A1M8 for 2D mapping, and proximity sensors for collision avoidance, enhancing its perception and interaction capabilities.

3.1.2 Power and Connectivity

Powered by a 26 Wh lithium-ion battery, the TurtleBot 4 offers 2.5–4 hours of runtime and recharges in approximately 2.5 hours. It includes integrated power management to monitor usage and ensure efficient operation.

3.1.3 Software Environment

Pre-installed with ROS 2, the TurtleBot 4 provides tools for navigation, mapping, and multi-robot coordination. Simulation support via Gazebo and user-friendly interfaces like RViz and rqt streamline development and testing.

3. Used tools and hardware

Table 3.1: Technical Specifications of TurtleBot 4

Component	Specifications
Mobile Base	iRobot Create 3
Drive System	Differential Drive Wheels
Payload Capacity	Up to 9 kg
Computing Unit	Raspberry Pi 4B (4 GB RAM)
Operating System	Ubuntu 20.04
ROS Version	ROS 2
Camera	OAK-D Pro Stereo AI Camera
LiDAR	RPLIDAR A1M8 2D LiDAR
Battery	26 Wh Lithium-Ion
Connectivity	USB 3.0, USB-C, Ethernet, Wi-Fi
Sensors	Proximity (IR and Bump), IMU, Encoders

3.1.4 Applications

Autonomous Navigation: Supports path planning, SLAM, and real-time obstacle avoidance for complex environments.

Research and Development: Ideal for developing algorithms in robotics, multi-robot systems, and AI-driven applications.

Education and Prototyping: Provides an accessible platform for learning robotics concepts and building custom solutions.

3.2 ROS2 (Robot Operating System 2)

ROS2 (Robot Operating System 2) is the second generation of the ROS framework, an open-source platform designed to streamline the development of robotic applications. ROS2 provides a comprehensive suite of libraries and tools that integrate sensors, actuators, and control modules into a unified operating environment. It addresses the limitations of ROS1, including improved support for distributed systems, real-time capabilities, and enhanced

scalability and reliability for modern robotic deployments.

3.2.1 Enhanced Real-Time Performance

ROS2 significantly improves **real-time performance** compared to ROS1, making it suitable for applications requiring high temporal precision. Key enhancements include:

- **RTOS Integration:** Seamlessly integrates with Real-Time Operating Systems, ensuring deterministic task scheduling and execution, essential for applications like robotic surgery.
- **Real-Time Threads Support:** Better support for real-time threads allows the execution of time-critical tasks without significant delays.
- **Improved Scheduling Mechanisms:** Advanced scheduling algorithms prioritize high-priority tasks, reducing latency in control loops.

These improvements make ROS2 ideal for precision movement control, real-time sensor feedback processing, and rapid obstacle avoidance in safety-sensitive applications.

3.2.2 Modular and Flexible Architecture

ROS2's **modular architecture** promotes flexibility, scalability, and ease of maintenance through a distributed system of nodes, each handling specific functionalities:

- **Decoupled Components:** Functions such as perception, navigation, and manipulation are implemented as separate nodes, allowing independent modification or replacement without affecting the entire system.
- **Scalability:** Supports scaling from single robots to large fleets, facilitating the addition of new robots and functionalities with minimal reconfiguration.

3. Used tools and hardware

- **Interoperability:** Ensures seamless integration between different modules and components, accommodating diverse hardware and software elements.
- **Namespace Management:** Enhanced namespace management improves organization and isolation of nodes, simplifying the management of complex systems.

This modularity simplifies development and debugging while fostering component reuse across different projects, reducing development time and costs.

3.2.3 Security Enhancements

In modern robotic applications, **security** is crucial to protect sensitive data and ensure safe operations. ROS2 addresses security through:

- **Encryption:** Encrypts all data transmitted between nodes, preventing unauthorized access and data breaches.
- **Authentication:** Verifies the identity of nodes within the network, ensuring only trusted entities can communicate.
- **Access Control:** Implements granular access policies, allowing administrators to define which nodes can publish or subscribe to specific topics.
- **Secure Communication Channels:** Utilizes DDS's secure communication features to establish trusted and tamper-proof channels between robots and controllers.

These measures are particularly important in industrial automation and healthcare robotics, where data integrity and operational safety are paramount.

3.2.4 Sensor Integration and Data Management

ROS2 facilitates efficient **sensor integration** and **inter-node communication**, enabling seamless management of data streams from diverse sources:

- **Sensor Fusion:** Combines data from multiple sensors (e.g., cameras, LiDAR, IMUs) to create a comprehensive environmental model, enhancing localization and perception.
- **Distributed Data Processing:** Enables parallel processing of sensor data across multiple nodes, improving computational efficiency and reducing latency.
- **Real-Time Data Handling:** Processes high-frequency sensor data in real-time, supporting applications that require immediate responses.
- **Standardized Interfaces:** Provides standardized interfaces and message formats for various sensors, simplifying integration and ensuring compatibility.

This robust sensor integration is essential for tasks like SLAM (Simultaneous Localization and Mapping), object recognition, and real-time decision-making.

3.2.5 Key Packages and Tools

ROS2 includes several packages and tools that enhance its functionality and usability. Notable among these are:

MoveIt 2

MoveIt 2 is a powerful motion planning framework integrated with ROS2, designed for complex manipulation tasks. Key features include:

- **Motion Planning:** Implements advanced algorithms for planning robot arm trajectories, ensuring smooth and collision-free movements.

3. Used tools and hardware

- **Kinematics and Dynamics:** Provides tools for calculating forward and inverse kinematics, enabling precise control of robotic manipulators.
- **Simulation Integration:** Seamlessly integrates with simulation environments like Gazebo, allowing virtual testing and validation of manipulation tasks.

MoveIt 2 is essential for applications such as industrial automation, service robotics, and research involving complex manipulation tasks.

Perception Packages

ROS2 offers a range of perception packages that enhance a robot's ability to interpret and understand its environment. Key packages include:

- **Point Cloud Library (PCL):** Provides tools for processing 3D point cloud data, enabling tasks like object detection, segmentation, and surface reconstruction.
- **OpenCV Integration:** Facilitates image processing and computer vision tasks, supporting applications such as object recognition, tracking, and semantic segmentation.
- **TensorFlow and PyTorch Integration:** Allows the integration of machine learning models, enabling advanced AI-driven perception capabilities.

3.2.6 RViz

RViz is a versatile 2D and 3D visualization tool integral to the ROS and ROS2 frameworks. It enables visualization of data from various robot sensors and nodes, providing a graphical representation of the surrounding environment. This is particularly useful for understanding robot behavior, verifying sensor data acquisition, and debugging robotic applications.

Core Features

RViz offers an intuitive interface for real-time visualization of diverse data types:

- **Maps:** Visualizes occupancy grids and topological maps generated during exploration and mapping, aiding in verifying map accuracy and adjusting sensor or movement models.
- **Point Clouds:** Displays 3D point cloud data from LiDAR or depth cameras for detailed environmental perception and obstacle detection.
- **Robot Models:** Shows the robot's physical model, including current pose, orientation, and state, providing a real-time view of movements and actions.
- **Sensor Data:** Visualizes raw and processed sensor data, such as camera feeds, IMU readings, and laser scans, facilitating real-time monitoring and analysis.
- **Interactive Markers:** Allows users to interact with the robot's environment by setting waypoints, adjusting target positions, or manipulating objects through a drag-and-drop interface.
- **Multi-Layer Visualization:** Supports simultaneous visualization of multiple data layers, including occupancy maps, point clouds, camera feeds, and planned trajectories, offering a comprehensive view of the environment and operational status.

Applications of RViz

RViz is a crucial tool in various aspects of robotic development and deployment. Its primary applications include:

- **Map Visualization:** RViz displays occupancy grids and topological maps generated during exploration and mapping, enabling developers to verify map accuracy and adjust sensor or movement models.

- **Real-Time Debugging:** Visualizing sensor readings like LiDAR or camera data in real-time helps diagnose and resolve issues during autonomous navigation by providing insights into the robot’s orientation, detected obstacles, and planned paths.
- **Path Visualization:** Paths calculated by the **Nav2** package are visualized in RViz, facilitating a clear understanding of the robot’s behavior and validating path planning algorithms. Users can observe how the robot navigates, avoids obstacles, and reaches various targets.
- **Interactive Marker Tools:** RViz includes interactive marker tools that allow users to set waypoints or adjust the robot’s target positions through a simple drag-and-drop interface, useful for testing new navigation strategies or manually guiding the robot in complex scenarios.
- **Comprehensive Visualization:** RViz supports the simultaneous display of multiple information layers, including occupancy maps, 3D point clouds, real-time camera feeds, and planned trajectories, providing a holistic view of the robot’s environment and operational status.
- **Customizable Displays:** Users can tailor the types of data displayed in RViz, such as sensor outputs, robot transforms, or navigation goals, allowing developers to focus on the most relevant data for their specific applications, streamlining the debugging and development process.

3.3 Libraries Used

In developing our robotic system, several external libraries have been integrated to enhance functionality, improve performance, and facilitate complex operations. This section provides an overview of the primary libraries utilized—**OpenCV** and **NetworkX**—along with their specific roles and applications within the project. Additionally, auxiliary libraries supporting clustering, spatial analysis, and morphological operations are discussed.

3.3.1 OpenCV

OpenCV (Open Source Computer Vision Library) is a widely-used open-source library for computer vision and machine learning, offering a vast collection of tools for image processing. Initially developed by Intel, OpenCV excels in efficiency and versatility, making it indispensable for manipulating visual data such as images and video streams. In robotics, OpenCV is essential for pre-processing maps, especially occupancy grid maps, enabling tasks like skeletonization, object detection, and map conversion.

Key functionalities include filtering, binarization, morphological operations, and edge detection, which are crucial for preparing maps for navigation. For example, filtering removes noise from sensor data, binarization simplifies occupancy maps by differentiating free and occupied cells, and morphological operations (erosion and dilation) refine maps by enhancing connectivity and eliminating small artifacts. These processes are vital for generating accurate and navigable environment representations.

OpenCV is also instrumental in generating Voronoi diagrams and their skeletonization, which help identify safe navigation paths and simplify map representations for path planning. Furthermore, OpenCV supports feature matching and object recognition, enhancing a robot's perception of its environment. Integration with deep learning frameworks like TensorFlow and PyTorch allows for advanced tasks such as semantic segmentation and object detection, enabling robots to adapt to dynamically changing environments.

Its compatibility with multiple programming languages, including Python and C++, and seamless integration with ROS2 nodes facilitate high-performance image processing and real-time data management, which are critical for reliable autonomous navigation and multi-robot collaboration.

3.3.2 NetworkX

NetworkX is a powerful Python library for creating, manipulating, and analyzing complex networks of nodes and edges. Widely used in various fields,

3. Used tools and hardware

NetworkX is crucial in robotics for representing spatial environments as graphs, enabling efficient navigation, path planning, and exploration by modeling key locations as nodes and navigable paths as edges.

NetworkX offers extensive functionalities, including the creation of directed and undirected graphs, shortest path calculations, clustering, and traversal algorithms like depth-first search (DFS) and breadth-first search (BFS). In our navigation system, NetworkX is utilized to manage the topological representation of the environment, facilitating structured navigation and reducing complexity compared to grid-based maps.

Key features include:

- **Graph Algorithms:** Supports algorithms for connected components, Eulerian paths, and minimum-cost flows, which are essential for solving the Chinese Postman Problem (CPP) and optimizing traversal paths.
- **Customization:** Allows for the integration of custom algorithms and modification of existing ones to meet specific application needs.
- **Graph Attributes:** Enables the addition of node and edge attributes, such as coordinates and weights, which are fundamental for distance calculations and path optimization.
- **Serialization:** Facilitates saving and loading graph structures using formats like JSON, ensuring easy modifications and expansions.
- **Integration with Python Ecosystem:** Compatible with libraries like SciPy and NumPy, enhancing its utility for complex calculations and optimizations.

NetworkX's ability to incorporate various graph attributes and integrate with other Python tools allows for efficient handling of spatial data and supports real-time path planning and multi-robot coordination, bridging high-level graph theory with low-level robot control.

3.3.3 Additional Libraries

Beyond OpenCV and NetworkX, several auxiliary libraries play supportive roles in data processing, clustering, and spatial analysis:

- **Scikit-Learn (sklearn):** Provides machine learning algorithms and tools for data analysis, including clustering algorithms like KMeans and DBSCAN, essential for identifying environmental patterns.
- **SciPy (scipy):** Offers modules for optimization, integration, interpolation, and spatial transformations. Utilized for tasks such as convolution, distance transformations, and filtering operations.
- **Scikit-Image (skimage):** Contains algorithms for image processing, including morphological operations like skeletonization, crucial for refining Voronoi diagrams and simplifying map representations.
- **NumPy (numpy):** Supports large, multi-dimensional arrays and matrices, along with mathematical functions for efficient numerical computations, fundamental for handling and processing sensor data.

These libraries complement OpenCV and NetworkX by providing robust tools for data manipulation, analysis, and visualization, thereby enhancing the overall functionality and efficiency of the robotic system.

Chapter 4

Original Contribution to the Solution of the Problem

The development of efficient and scalable solutions for multi-robot navigation and coordination is a complex challenge that combines diverse aspects of robotics, graph theory, and software engineering. In this chapter, we present our original contributions to addressing these challenges by focusing on the generation of topological maps, robust navigation, and multi-robot coordination.

Our proposed system leverages advanced techniques to transform occupancy grid maps into high-quality topological representations, enabling efficient task allocation and scalable multi-robot exploration. By incorporating collision-aware graph construction, skeletonization, and Voronoi diagram techniques, we create compact yet highly representative navigational structures. Additionally, we enhance traditional graph traversal algorithms, such as the Chinese Postman Problem (CPP), to meet the specific needs of robotic systems.

In the context of multi-robot coordination, we introduce an edge-centric resource management strategy and a dynamic master-slave framework. These innovations address key issues such as collision avoidance, edge utilization optimization, and fault tolerance through robust communication protocols and a refined master election mechanism. Moreover, the system seamlessly

4. Original Contribution to the Solution of the Problem

integrates with the ROS2 navigation stack to achieve high-frequency edge traversal and robust navigation in real-world environments.

This chapter details the methodologies and implementation strategies underlying our contributions, demonstrating how they collectively advance the state of the art in robotic navigation and coordination. Through rigorous design and integration of scalable, fault-tolerant architectures, our work lays a solid foundation for future advancements in autonomous robotics.

4.1 High-Level System Operation

In this section, we provide a high-level overview of how our implemented system operates to enable efficient multi-robot navigation and coordination. The overall architecture is centered on transforming occupancy grid map data into a topological representation that is subsequently used to guide and manage the robot(s) in a dynamic environment.

4.1.1 Pipeline Overview

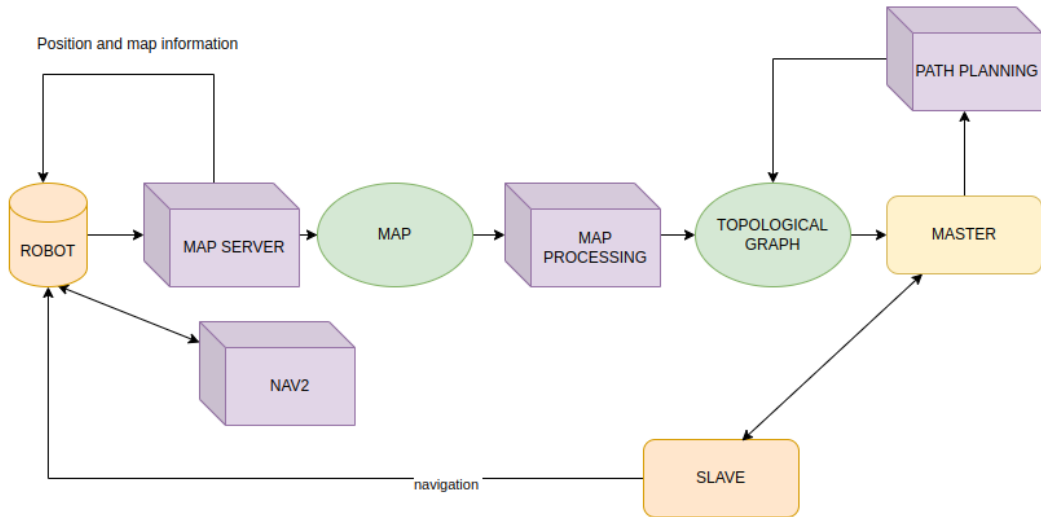


Figure 4.1: Pipeline overview describing the developed architecture

4. Original Contribution to the Solution of the Problem

The system architecture, depicted in Figure 4.1, involves retrieving a pre-built two-dimensional occupancy grid map from the map server, processing it to create a topological graph, and utilizing this graph for path planning and optimization. The master node computes optimal paths and assigns them as waypoints to the slave robots, which navigate the environment accordingly.

1. **Map Acquisition:** The robot receives a pre-built two-dimensional occupancy grid map from the map server over the network. It uses its onboard sensors to localize itself within the map, ensuring accurate navigation and positioning.
2. **Map Processing:** The map processing module retrieves the occupancy grid map and applies skeletonization and Voronoi-based techniques to generate a topological graph, consisting of:
 - *Nodes*: Representing navigable regions or critical points within the environment.
 - *Edges*: Connecting these nodes and indicating feasible navigation paths.
3. **Graph Refinement:** A collision-aware mechanism refines the topological graph by pruning unsafe edges and adjusting the graph to account for obstacles and dynamic changes in the environment.
4. **Path Planning:** The path planning module computes optimal trajectories for each slave robot by leveraging the refined topological graph. Edge-centric metrics are calculated during this process to optimize resource utilization, ensuring efficient task allocation, minimizing potential conflicts, and enhancing the reliability and scalability of multi-robot operations.
5. **Waypoint Assignment:** The computed optimal paths are translated into waypoints, which are assigned to the respective slave robots, guiding them effectively through the environment.

4.1.2 Master-Slave Coordination

Once the topological graph is constructed, the system identifies a *master* robot through a dynamic election mechanism. The master assumes responsibility for:

1. **Global Path Planning:** Computing optimal trajectories for each *slave* robot based on the topological graph structure and task requirements.
2. **Edge Allocation:** Assigning edges to slave robots in a manner that prevents overlap and potential collisions.
3. **Deadlock and Collision Avoidance:** Monitoring the positions of all active robots, detecting potential bottlenecks, and resolving deadlock situations by dynamically rerouting or adjusting traversal schedules.

4.1.3 Slave Robot Execution

The slave robots receive their assigned paths from the master and proceed to execute them using the integrated navigation stack. Their responsibilities include:

- Following the prescribed topological path while continuously updating the master regarding their status.
- Reacting to local environmental changes (e.g., unexpected obstacles) by adjusting their immediate path, without compromising the global path plan.
- Maintaining reliable communication channels to report progress, resource usage, and any anomalies that could lead to potential conflicts.

4.1.4 Overall Workflow

Putting all these components together, the sequence of operations for each mission can be summarized as:

4. Original Contribution to the Solution of the Problem

1. **Initialization:** The robot starts by collecting and sharing the occupancy grid map.
2. **Graph Generation:** The map is converted into a topological graph with collision-aware edges.
3. **Master Election:** There is a master, responsible for global path planning and coordination.
4. **Task Allocation & Execution:** The master computes optimal paths for the slave robots, assigns edges, and oversees traversal.
5. **Monitoring & Adaptation:** Both the master and slaves continuously monitor for collisions, deadlocks, and environmental changes, adapting routes where necessary. This functionality is achieved through the integration of the map server and the *Nav2* stack, which are already implemented within the ROS2 architecture. Leveraging these robust, pre-existing components eliminates the need for re-implementing these capabilities, allowing the system to focus on higher-level coordination and optimization tasks.

This multi-layered strategy ensures that our system remains robust, flexible, and scalable in complex and dynamic scenarios. By distributing tasks intelligently and continuously updating navigation plans, we achieve a highly efficient coordination framework that significantly reduces the risk of deadlocks and collisions.

4.2 Topological Map Generation from an Occupancy Grid Map

Generating a topological map from an *Occupancy Grid Map (OGM)* is a fundamental step toward efficient navigation and path planning in robotic systems. Although OGMs provide a detailed representation by

4. Original Contribution to the Solution of the Problem

storing free, occupied, or unknown status for each cell, they can become computationally heavy when dealing with large environments or multi-robot setups. Furthermore, OGMs often include superfluous information such as small protrusions or isolated obstacles that have negligible impact on navigable paths.

By contrast, topological maps abstract the environment into a network of nodes and edges that emphasize navigable routes and connectivity over high-resolution detail. This approach significantly reduces computational overhead and enhances scalability, especially in dynamic and heterogeneous environments.

4.2.1 Modular Structure of the Codebase

The implementation adopts a highly modular architecture, facilitating code readability, maintainability, and extensibility. Each module is designed to handle a specific aspect of the map processing pipeline, as shown in Figure 4.2. The key modules are as follows:

- **CoordinateTransformer:** Handles the transformation between real-world coordinates and image pixel coordinates, ensuring accurate spatial mapping.
- **ImageProcessing:** Performs all map-related operations, including loading, cleaning, binarization, distance map calculation, and skeletonization. This module ensures that the map is optimally prepared for graph creation.
- **GraphCreation:** Constructs the topological graph from the skeletonized map by identifying nodes and edges, while ensuring connectivity and collision-free pathways.
- **Visualization:** Handles graph and map visualization by overlaying nodes, edges, and labels onto the original map and saving the output in various formats.

4. Original Contribution to the Solution of the Problem

The modularity ensures that each component can be independently tested, updated, or replaced without affecting other parts of the pipeline, making the system robust and scalable.

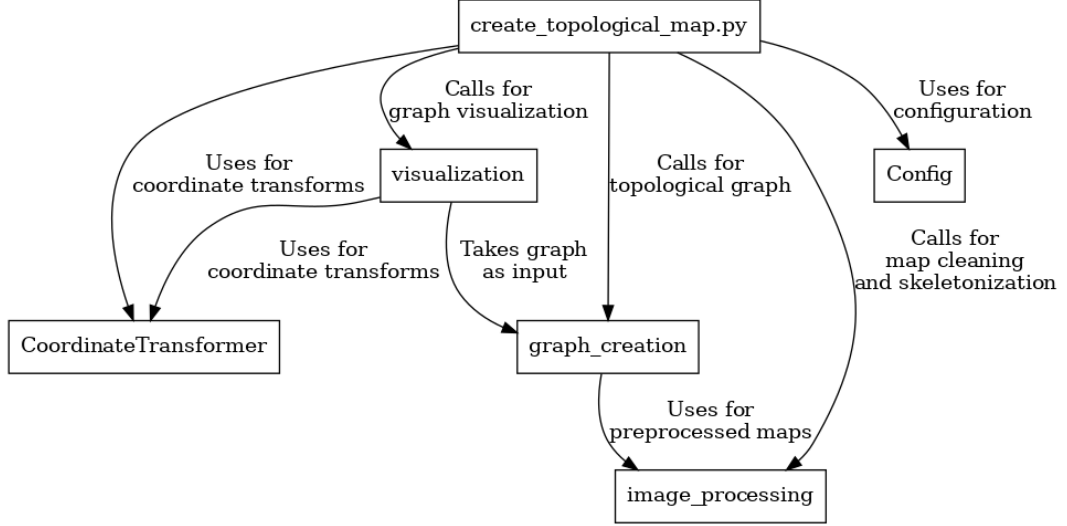


Figure 4.2: Diagram describing the modular code implementation used during the map processing pipeline.

4.3 Map Cleaning and Preparation

The transformation from an OGM to a topological map requires a careful cleaning stage. This preprocessing step ensures that only relevant structural elements are preserved, thereby increasing the efficiency and accuracy of subsequent topological analysis.



Figure 4.3: Processed Occupancy Grid Map after noise reduction and removal of minor obstacles, highlighting essential structures for topological mapping.

4.3.1 Morphological Cleaning Pipeline

A series of morphological operations is applied to the original OGM to reduce noise and highlight meaningful features:

1. **Morphological Closing:** Combines dilation followed by erosion using a rectangular kernel. It fills small gaps and bridges fragmented regions, delivering a more continuous representation of open spaces.
2. **Dilation:** Expands the boundaries of prominent features to improve connectivity across adjacent structures. Minor irregularities are smoothed out, reinforcing the primary navigable areas.
3. **Morphological Opening:** Consists of erosion followed by dilation. It removes small noise elements while preserving key structural components, ensuring navigable paths remain intact.
4. **Erosion:** Contracts feature boundaries to restore their shapes and eliminate residual noise. This final step ensures the structural fidelity needed for robust topological analysis.

Each operation uses adaptively sized kernels, derived from parameters such as the map resolution and a `min_feature_size` threshold. This adaptive approach balances noise suppression against the need to retain crucial structures.

4.3.2 Outcome and Applications

The cleaned map provides a reliable foundation for subsequent topological methods, including:

- **Voronoi Diagram Creation:** Partitions the navigable space into regions that are equidistant to the closest obstacles, thus highlighting pathways that maximize safety.

- **Skeletonization:** Reduces the navigable areas to a simplified skeletal structure, retaining connectivity and topology. It serves as a more efficient graph representation for planning algorithms.

This simplified, high-level representation of the environment supports efficient task allocation, scalable multi-robot coordination, and robust path planning.

4.4 Skeletonization and Voronoi Diagram Techniques

4.4.1 Binary Map Generation

After map cleaning, thresholding is employed to convert the grayscale map into a binary format. Each pixel is classified as free (white) or occupied (black), thus streamlining further distance-transform and skeletonization computations.



Figure 4.4: Binary Map derived from the cleaned Occupancy Grid Map, distinguishing free space (white) from obstacles (black).

4.4.2 Distance Map Calculation

A *Distance Map* is created using the Euclidean Distance Transform (EDT). In this stage, each free pixel is assigned a value corresponding to its

4. Original Contribution to the Solution of the Problem

Euclidean distance to the nearest obstacle. The map is then normalized for consistency across environments.

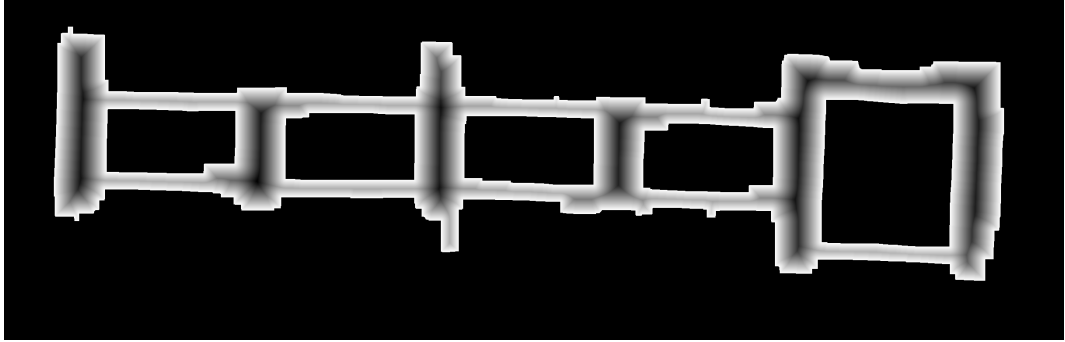


Figure 4.5: Normalized Distance Map obtained via EDT, illustrating the proximity of each free pixel to the nearest obstacle.

4.4.3 Skeletonization

Skeletonization directly applies a thinning algorithm to the normalized distance map, highlighting the most critical navigational routes. This step reduces navigable areas to one-pixel-wide paths without losing key topological connections, allowing for a more efficient graph-based planning approach.

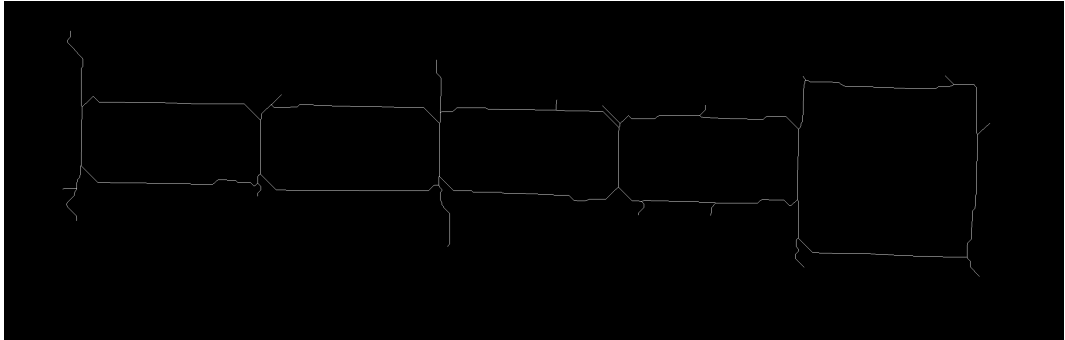


Figure 4.6: Skeletonized Voronoi Map, showing fundamental pathways that enable graph-based navigation.

Algorithmic Details

Voronoi Diagram Construction: Identifies sets of points equidistant to obstacles, effectively delimiting corridors and open spaces suited for safe navigation.

Euclidean Distance Transform (EDT): Assigns a distance value to each free pixel, vital for both skeletonization and Voronoi diagram generation.

These techniques together facilitate an abstract but navigable representation that streamlines path planning and multi-agent coordination.

4.4.4 Implementation Considerations

All algorithms in the codebase can be adapted to different environments and resolutions. Key parameters (e.g., kernel sizes) are computed in real time based on `min_feature_size` and the map's resolution, ensuring consistent outcomes across a diverse set of use cases.

4.5 Algorithmic Implementation Overview

4.5.1 Morphological Operations

Morphological operations remove noise and highlight critical features. They are tuned dynamically to preserve essential structures at various map scales. This includes:

- **Closing:** Fills small gaps.
- **Dilation:** Improves feature connectivity.
- **Opening:** Removes spurious artifacts.
- **Erosion:** Restores boundary integrity.

4.5.2 Euclidean Distance Transform (EDT)

This subsection explores the `distance_transform_edt` function, detailing its operational principles, algorithmic foundation, and its essential role in the topological map generation pipeline.

Understanding the Euclidean Distance Transform

The Euclidean Distance Transform (EDT) calculates, for each pixel in a binary image, the shortest Euclidean distance to the nearest non-zero (occupied) pixel. In occupancy grid maps, free spaces are represented by zero or low-intensity values, while obstacles are marked by non-zero or high-intensity values. EDT assigns each free pixel a value corresponding to its Euclidean distance from the nearest obstacle, effectively outlining navigable regions.

Mathematically, for a binary image I , the EDT D is defined as:

$$D(x, y) = \min_{(i,j) \in \Omega} \sqrt{(x - i)^2 + (y - j)^2}$$

where Ω is the set of all obstacle pixels, and (x, y) are the coordinates of a free pixel.

Functionality of `distance_transform_edt`

Input Preparation Before applying EDT, the occupancy grid map undergoes:

1. *Binary Conversion*: Ensures the map is in a binary format suitable for EDT.
2. *Noise Reduction*: Applies morphological operations to eliminate spurious noise and small artifacts that could distort distance calculations.

Algorithmic Steps The `distance_transform_edt` function operates through the following phases:

1. *Initialization*: A distance matrix D matching the input image dimensions is initialized. Obstacle pixels are set to zero, and free pixels are assigned a large value (e.g., infinity).
2. *Forward Pass*: Iterates from top-left to bottom-right. For each free pixel (x, y) , it updates $D(x, y)$ based on the minimum distance of the

4. Original Contribution to the Solution of the Problem

top, top-left, and top-right neighbors plus their respective Euclidean distances.

3. *Backward Pass*: Iterates from bottom-right to top-left. For each free pixel (x, y) , it updates $D(x, y)$ using the bottom, bottom-left, and bottom-right neighbors similarly to the forward pass.
4. *Finalization*: After both passes, D contains accurate Euclidean distances for all free pixels. The distance map is then normalized to a standard scale (e.g., 0 to 255) for visualization and consistency across environments.

Normalization Process Normalization scales the raw distance values to a uniform range, typically $[0, 255]$, aligning with standard image intensity ranges. This enhances the interpretability of the distance map and ensures effective subsequent image processing steps, such as skeletonization.

Normalization is performed using:

$$D_{normalized}(x, y) = \left(\frac{D(x, y)}{D_{max}} \right) \times 255$$

where D_{max} is the maximum distance value in the distance map. This scaling ensures that the farthest free pixels from any obstacle receive the highest intensity values, visually representing open and safe navigable regions.

4.5.3 Skeletonization

Skeletonization is a crucial image processing technique that reduces binary images to their essential structural form. By iteratively thinning the boundaries of free-space regions, it transforms complex navigable areas into simplified, one-pixel-wide skeletal structures. This process preserves the topological and geometric properties of the original map, ensuring that critical navigational pathways remain intact while eliminating redundant details.

Fundamental Principles

The primary objective of skeletonization is to extract the **medial axis** of shapes within a binary image. The medial axis consists of all points equidistant to multiple boundary points, representing central corridors and pathways that define the environment's topology.

Key characteristics of the skeleton include:

- **Connectivity Preservation:** Maintains the same navigational pathways as the original free-space regions.
- **Dimensionality Reduction:** Simplifies the representation by reducing navigable space to a minimalistic graph-like structure without significant loss of information.

Algorithmic Approach

We employ the **Zhang-Suen Thinning Algorithm** for skeletonization, which iteratively removes boundary pixels while preserving topological features. The algorithm operates on a binary image where foreground pixels (value 1) represent navigable spaces and background pixels (value 0) denote obstacles.

Steps of the Zhang-Suen Thinning Algorithm

1. **Initialization:** Create a distance matrix D matching the input image dimensions. Set obstacle pixels to zero and free pixels to a large value (e.g., infinity).
2. **Forward Pass:** Traverse the image from top-left to bottom-right. For each free pixel $P(x, y)$, evaluate its eight neighbors. Mark P for removal if it satisfies:
 - It has between 2 and 6 foreground neighbors.
 - It has exactly one transition from background to foreground in a clockwise traversal of its neighbors.

4. Original Contribution to the Solution of the Problem

- It has at least two specific background pixels among its neighbors to preserve connectivity.
3. **Backward Pass:** Traverse the image from bottom-right to top-left. Repeat the evaluation with symmetrical conditions to ensure uniform thinning from all sides.
 4. **Removal:** Remove all marked pixels simultaneously after both passes.
 5. **Iteration:** Repeat the forward and backward passes until no more pixels can be removed without violating the constraints.

The result is a one-pixel-wide skeleton that accurately represents the medial axis of the navigable space.

Preservation of Topology

The Zhang-Suen algorithm preserves topology by enforcing rules that prevent disconnections and maintain structural integrity. Pixels are only removed if their elimination does not fragment connected components or alter critical features such as loops, corridors, or intersections. This ensures that the skeleton accurately reflects the navigable structure of the environment while discarding unnecessary details.

4.5.4 Topological Graph Creation

From the skeletonized Voronoi map, the system identifies nodes (intersections, dead-ends) and merges those in close proximity using DBSCAN to reduce redundancy. Edges are then established between nodes by checking for navigability along connecting lines.

4.5.5 Coordinate Transformation

This step aligns the final topological graph with real-world coordinates, essential for deployment on physical robots. The `min_feature_size` parameter affects coordinate scaling to maintain spatial fidelity in real environments.

4. Original Contribution to the Solution of the Problem

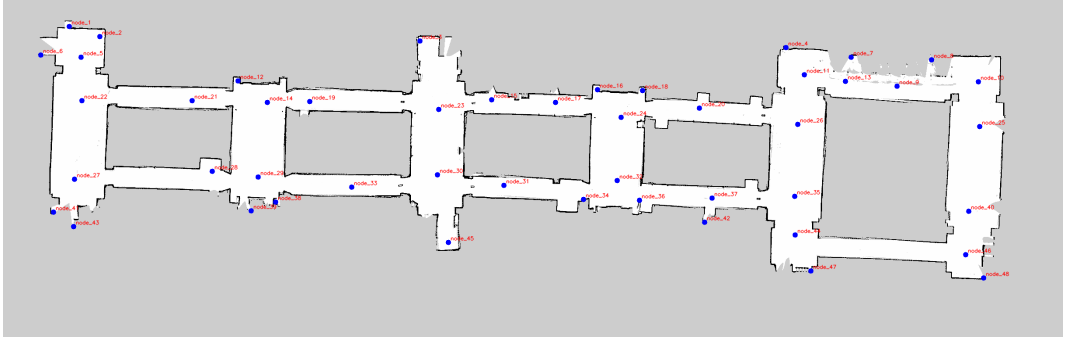


Figure 4.7: Topological graph overlaid on the original Occupancy Grid Map, showing nodes accurately mapped to real-world coordinates.

4.6 Topological Graph Construction Using Collision-Aware Techniques

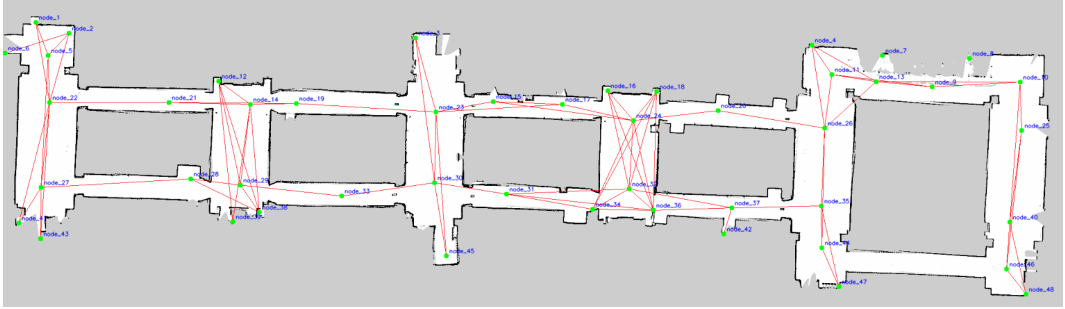


Figure 4.8: Topological graph superimposed on the original Occupancy Grid Map, displaying nodes and edges that define navigable routes.

4.6.1 Overview

A **collision-aware topological graph** is generated to ensure reliable navigation. Only node-to-node paths that pass a collision-free check are retained. This produces a simplified yet representative map of the environment's traversable routes.

4.6.2 Graph Construction

- **Node Initialization:** An initial set of waypoints is defined according to known coordinates. Each waypoint is transformed from metric units to pixel coordinates to align with the underlying occupancy grid.
- **Collision Checking:** Potential connections between nodes are validated by analyzing the proportion of free cells along the connecting line in the occupancy grid. If it exceeds a certain threshold, the edge is considered collision-free.
- **Edge Limitation:** Each node limits its connections to a subset of its nearest or most relevant neighbors, preventing the graph from exploding in size.

4.6.3 Connecting Disconnected Components

If the resulting graph contains disconnected clusters of nodes:

1. **Component Identification:** A connectivity analysis is performed to detect isolated groups of nodes.
2. **Candidate Selection:** Additional pairs of nodes from different components are considered, factoring in distance minimization, collision avoidance, and balanced connectivity.
3. **Edge Addition:** Valid connections are incrementally added until either the graph is fully connected or no further connections satisfy the criteria.

4.6.4 Graph Representation

The final graph comprises:

- **Nodes:** Representing waypoints in both pixel and real-world coordinates.

4. Original Contribution to the Solution of the Problem

- **Edges:** Collision-free links between nodes annotated with Euclidean distances, enabling path-length calculations.

Such an approach is advantageous for:

- **Safety:** Ensures all routes are obstacle-free.
- **Scalability:** Reduces complexity by limiting the number of edges and iteratively merging disconnected components.
- **Robustness:** Guarantees that all navigable areas are reachable, maintaining a reliable topological structure.

4.7 Design and Choice of Navigation Algorithm

Efficient navigation and coordination in multi-robot systems require algorithms that can ensure comprehensive coverage of all edges in a graph while minimizing traversal costs. After evaluating multiple approaches, we selected the **Chinese Postman Problem (CPP)** algorithm due to its ability to compute an optimal Eulerian circuit for traversing every edge of a graph at least once. This section outlines the rationale behind our choice and describes how the algorithm was adapted and integrated into the system.

4.7.1 Choice of Algorithm

The decision to adopt the CPP was based on a thorough evaluation of the system's requirements and the available alternatives. Below, we discuss the advantages and limitations of the CPP compared to other algorithms commonly used in graph traversal:

- **Chinese Postman Problem (CPP):**

- **Pros:**

4. Original Contribution to the Solution of the Problem

- * Directly addresses edge traversal, ensuring that all edges in the graph are visited with minimal additional cost.
- * Computationally efficient with a polynomial-time complexity, making it suitable for real-time applications.
- * Generates an Eulerian circuit in transformed graphs, ensuring seamless navigation in environments represented by topological maps.

– **Cons:**

- * Assumes a static graph, requiring periodic updates or reevaluation to handle dynamic environments.

- **Travelling Salesman Problem (TSP):**

– **Pros:**

- * Finds the shortest path that visits all nodes, which can be useful when node visitation (e.g., task-specific locations) is the primary objective.
- * Can be adapted to edge traversal by modifying the graph structure, though at the expense of increased complexity.

– **Cons:**

- * NP-hard problem, resulting in significantly higher computational costs for larger graphs.
- * Focuses on nodes rather than edges, making it less efficient for our specific application where edge traversal is the primary goal.

- **Depth-First Search (DFS) and Breadth-First Search (BFS):**

– **Pros:**

- * Simple to implement and computationally inexpensive, making them ideal for basic exploration tasks.
- * Ensure complete coverage of the graph in terms of visiting all nodes.

4. Original Contribution to the Solution of the Problem

- **Cons:**

- * Do not guarantee optimal edge traversal, potentially leading to redundant or inefficient paths.
- * Lack mechanisms to minimize traversal costs or revisit edges systematically.

- **Modified Eulerian Traversal Algorithms:**

- **Pros:**

- * Efficiently compute Eulerian circuits in naturally Eulerian graphs.
- * Suitable for scenarios where graphs already meet Eulerian conditions (all vertices have even degrees).

- **Cons:**

- * Require additional preprocessing for non-Eulerian graphs, similar to the CPP.
- * Lack the flexibility of CPP in minimizing traversal costs when additional edges are required.

Based on this comparison, the CPP was deemed the most appropriate choice for our system. It effectively balances computational efficiency, optimal edge traversal, and compatibility with dynamic updates, addressing the unique challenges posed by multi-robot navigation in graph-based environments.

4.7.2 Design and Adaptation of the Algorithm

The successful implementation of the CPP algorithm required its adaptation to the specific requirements of our robotic navigation system. The following steps were taken during the design phase:

Graph Representation and Preprocessing

4. Original Contribution to the Solution of the Problem

- The input graph was derived from a skeletonized version of the cleaned occupancy grid map, representing nodes as waypoints and edges as navigable paths.
- For graphs that were not naturally Eulerian (i.e., with vertices of odd degree), additional edges were introduced to ensure all vertex degrees were even. This transformation was performed using the **Minimum Weight Matching (MWM)** approach to minimize the added traversal cost.
- A collision-aware preprocessing step ensured that only edges free from obstacles were retained, guaranteeing safe navigation for robots.

4.8 Introduction to the Chinese Postman Problem (CPP)

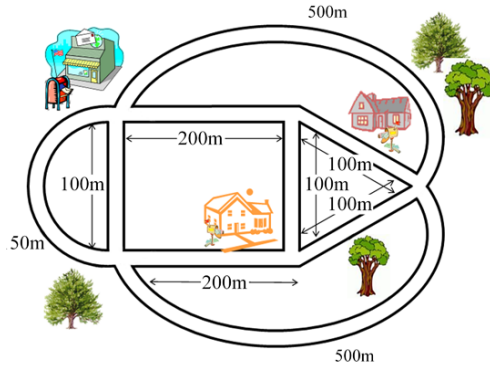


Figure 4.9: Chinese postman problem overview

The Chinese Postman Problem (CPP), also known as the route inspection problem, is a classic problem in graph theory and combinatorial optimization. It involves finding the shortest closed path or circuit in a graph that traverses every edge at least once. This problem has applications in logistics, urban planning, and network optimization.

Originally studied by the Chinese mathematician Meigu Guan in 1960, the problem's name honors his contributions. His work was translated into

4. Original Contribution to the Solution of the Problem

English in 1962, and the term "Chinese Postman Problem" was coined, attributed to Alan J. Goldman or Jack Edmonds at the U.S. National Bureau of Standards. Unlike the NP-hard Travelling Salesman Problem, the CPP is solvable in polynomial time, emphasizing its practical utility in various domains.

If the graph contains an Eulerian circuit—a closed walk covering every edge exactly once—that circuit is the optimal solution. Otherwise, the optimization involves adding edges to ensure all vertex degrees are even, transforming the graph into an Eulerian multigraph with minimal additional weight.

A generalization of the problem involves the concept of a T-join, where a subset of edges is chosen to ensure specific vertices (those with odd degrees) achieve even degrees. Finding the minimum-weight T-join is integral to solving the CPP.

4.9 Definition and Key Concepts

- **Eulerian Circuit:** If a graph has a closed walk that visits every edge exactly once, it is Eulerian. For such graphs, the Eulerian circuit is the optimal solution to CPP.
- **Odd-Degree Vertices:** If a graph is not Eulerian, the optimization involves identifying vertices with an odd degree and pairing them such that the total additional weight is minimized.
- **T-Join:** A T-join in a graph is an edge set where the vertices with an odd number of edges (odd-degree vertices) correspond exactly to the set T . Finding a minimal T-join is crucial in solving the CPP.

The undirected route inspection problem can be solved in polynomial time by leveraging the concept of a T-join. Let T be the set of vertices in the graph with odd degrees. An edge set J is called a T-join if the collection of vertices that have an odd number of incident edges in J

4. Original Contribution to the Solution of the Problem

matches exactly the set T . A T-join exists whenever every connected component of the graph contains an even number of vertices in T .

For any set T , the smallest T-join (if it exists) consists of $\frac{1}{2}|T|$ paths that pair up the vertices in T . The paths are chosen to minimize the total length or weight. In an optimal solution, these paths may share vertices but will not share edges.

To find the minimum T-join:

1. Construct a complete graph using the vertices in T , where the edge weights represent the shortest path distances between these vertices in the original graph.
2. Find a minimum-weight perfect matching in this complete graph. This matching corresponds to the paths needed in the original graph to form the T-join.
3. Adding the edges corresponding to this T-join to the original graph ensures all vertices have even degrees, transforming the graph into an Eulerian multigraph.

These operations, including constructing the complete graph and finding the matching, can be done in $O(n^3)$ computational steps.

4.9.1 Algorithm Overview

4.9.2 Undirected Graph Solution

For undirected graphs:

1. Identify all vertices with odd degrees (set T).
2. Compute shortest paths between all pairs of vertices in T .
3. Construct a complete graph on T , using shortest path distances as edge weights.
4. Find a minimum-weight perfect matching in this complete graph.

4. Original Contribution to the Solution of the Problem

5. Add the corresponding edges to the original graph to create an Eulerian multigraph.
6. Find an Eulerian circuit in the modified graph.

4.9.3 Directed Graph Solution

For directed graphs:

1. Ensure the graph is strongly connected. If it is not, no Eulerian circuit exists.
2. Calculate the imbalance of in-degree and out-degree for each vertex. Vertices with excess in-degree or out-degree are identified.
3. Solve a minimum-cost flow problem to balance in-degrees and out-degrees. In this formulation:
 - Each unit of excess in-degree is treated as supply.
 - Each unit of excess out-degree is treated as demand.
4. The minimum-cost flow problem is solvable in $O(|V|^2|E|)$ time. Adding the flow edges to the graph ensures all vertices have balanced in-degrees and out-degrees.
5. Determine the Eulerian circuit in the balanced directed graph.

4.9.4 Applications

The CPP has practical applications and has been used to reduce various combinatorial problems:

- **Urban Planning and Logistics:** Optimizing routes for mail delivery, garbage collection, snow plowing, and street sweeping.
- **Network Maintenance:** Designing efficient inspection routes for utilities such as power lines, pipelines, and communication networks.

- **Graph Theory Problems:** Finding a maximum cut in planar graphs or determining minimum-mean length circuits in undirected graphs.

4.9.5 Variants of the CPP

Several variants of the CPP have been explored:

- **Windy Postman Problem:** In this variant, traversal costs depend on the direction of traversal. The problem becomes NP-complete due to the directional asymmetry in edge costs.
- **Mixed Chinese Postman Problem:** Some edges are directed, while others are undirected. This variant is also known as the "New York Street Sweeper Problem."
- **k-Chinese Postman Problem:** This variant involves finding k circuits, each starting from a designated location, such that every edge is covered at least once. The goal is to minimize the cost of the longest circuit.
- **Rural Postman Problem:** In this variant, only a subset of edges must be visited. This problem is relevant in scenarios where not all streets or paths require traversal.

4.9.6 Solution focusing on Unirected Graph Solution

This section focuses on the approach implemented to solve the Chinese Postman Problem (CPP) for undirected graphs. The implementation involves adapting theoretical concepts into practical algorithms while addressing the specific requirements of our robotic navigation system. By leveraging advanced graph theory principles such as Minimum Weight Matching and Eulerian circuit construction, the solution ensures optimal edge traversal in the context of the environment represented by a topological map. The methods detailed here were chosen and tailored to align with the computational constraints and real-time demands of the robotic platform.

4.9.7 Understanding Matching in Graphs

A **matching** in a graph is a set of edges where no two edges share a common node. Matching is a fundamental concept in graph theory and has various types depending on the optimization criteria:

- **Maximum Matching:** A matching that includes the largest possible number of edges.
- **Maximum Weight Matching:** A matching where the total weight of the selected edges is maximized.
- **Minimum Weight Matching:** A matching where the total weight of the selected edges is minimized.
- **Perfect Matching:** A matching in which every node in the graph is incident to exactly one edge of the matching.

In our problem, we focus on finding a **Minimum Weight Matching**, which is crucial for minimizing the cost of adding edges to create an Eulerian multigraph.

4.9.8 Detailed Explanation of ‘min_weight_matching’ and ‘max_weight_matching’

NetworkX, the Python library used in our implementation, provides robust tools for solving matching problems. While ‘max_weight_matching’ is widely recognized, the availability of ‘min_weight_matching’ simplifies our task. Here, we delve into how these functions operate.

How ‘min_weight_matching’ Works

The ‘min_weight_matching’ algorithm computes a matching that minimizes the total weight of selected edges. Its implementation involves the following steps:

4. Original Contribution to the Solution of the Problem

1. **Graph Initialization:** The algorithm ensures the graph is undirected and checks for edge weights. If weights are missing, they default to 1.
2. **Edge Sorting:** The edges are sorted by weight in ascending order. This prioritization ensures that lighter edges are considered first during matching.
3. **Iterative Edge Selection:** Edges are added to the matching one at a time, provided they do not conflict with already selected edges (i.e., no shared nodes).
4. **Termination Condition:** The algorithm stops when no additional edges can be added without violating the matching constraints.

The use of efficient data structures ensures that the algorithm achieves a time complexity of $O(n^3)$ for dense graphs.

4.9.9 Algorithmic Guarantees and Practical Considerations

Algorithmic Efficiency

- **Time Complexity:** $O(n^3)$, suitable for dense graphs.
- **Scalability:** Effective for medium to large graphs, but performance can degrade for very large datasets.

4.9.10 Additional Insights on Eulerian Tour

Definition and Historical Context

An **Eulerian trail** (or Eulerian path) in a graph is a walk that visits every edge exactly once. Similarly, an **Eulerian circuit** (or Eulerian cycle) is an Eulerian trail that starts and ends at the same vertex. These concepts were introduced by Leonhard Euler in his famous solution to the *Seven Bridges of Königsberg* problem in 1736, laying the foundation of graph theory.

4. Original Contribution to the Solution of the Problem

Euler's key insight was the characterization of graphs that possess these properties:

- A connected graph has an **Eulerian circuit** if and only if every vertex has an even degree.
- A connected graph has an **Eulerian trail** if and only if exactly zero or two vertices have an odd degree.

Properties of Eulerian Graphs

- A graph with all vertices of even degree is called an **Eulerian graph** and necessarily contains an Eulerian circuit.
- A graph with exactly two vertices of odd degree is called **semi-Eulerian** and contains an Eulerian trail but not an Eulerian circuit.

Hierholzer's Algorithm for Eulerian Circuits Hierholzer's algorithm efficiently finds Eulerian circuits in connected graphs where all vertices have even degree:

1. **Initialization:** Start at any vertex and traverse edges until returning to the starting vertex, forming a closed tour. This step ensures all traversed edges are part of the Eulerian circuit.
2. **Expansion:** If unvisited edges remain, pick a vertex from the tour that has adjacent unvisited edges, and form a new closed tour starting and ending at that vertex.
3. **Joining Tours:** Integrate the new tour into the existing one by combining their shared vertices.
4. **Completion:** Repeat until all edges are included in the Eulerian circuit.

The algorithm runs in linear time, $O(|E|)$, using efficient data structures to manage unvisited edges and tours.

Connection to the Chinese Postman Problem (CPP) In the context of the CPP, the goal is to transform a graph into an Eulerian graph (where an Eulerian circuit exists) by adding the fewest additional edges. This transformation is achieved by addressing vertices with odd degrees, as described below:

- Identify all vertices with odd degrees (set T).
- Pair these vertices using a **minimum weight perfect matching** in a complete graph constructed from T , where edge weights correspond to shortest path distances in the original graph.
- Add the edges from this matching to the original graph, ensuring all vertices now have even degrees, thus forming an Eulerian multigraph.
- Apply Hierholzer's algorithm to determine the optimal Eulerian circuit.

By integrating these principles, the CPP is solved efficiently, minimizing the total additional weight required to enable an Eulerian circuit in the given graph.

4.9.11 Additional Insights on Matching Algorithms

Blossom Algorithm for Maximum Weight Matching

The **Blossom Algorithm**, developed by Jack Edmonds in 1961 and formally published in 1965, is a fundamental breakthrough in graph theory, designed to construct **maximum matchings** in general graphs. The algorithm identifies a matching M such that each vertex in the graph $G = (V, E)$ is incident with at most one edge in M , and the size of M ($|M|$) is maximized. Unlike algorithms tailored to bipartite graphs, the Blossom Algorithm introduces a novel mechanism to handle **odd-length cycles**, known as **blossoms**, through a process called **contraction**. This technique simplifies the graph by reducing an odd-length cycle into a single vertex, allowing the search for augmenting paths to continue iteratively in the contracted graph.

4. Original Contribution to the Solution of the Problem

The iterative process of the Blossom Algorithm begins with an initial empty matching and improves it by repeatedly finding and augmenting along alternating paths in the graph. This iterative refinement is key to constructing an optimal matching. The ability to manage odd-length cycles makes the algorithm uniquely effective for non-bipartite graphs, a feature that sets it apart from bipartite-specific algorithms like the Hungarian Algorithm.

The computational efficiency of the Blossom Algorithm is notable, with a time complexity of $O(|E||V|^2)$, where $|E|$ is the number of edges and $|V|$ is the number of vertices in the graph. While faster methods such as the Micali-Vazirani algorithm ($O(|E|\sqrt{|V|})$) exist, the Blossom Algorithm remains a cornerstone due to its conceptual clarity and practical versatility.

The algorithm's significance extends beyond its direct application to maximum matchings. It provided the first proof that a maximum-size matching could be found in polynomial time, marking a major milestone in combinatorial optimization. Additionally, it laid the groundwork for the linear programming polyhedral description of the **matching polytope**, enabling the development of algorithms for problems like **minimum weight matching**.

By introducing foundational concepts like contraction and leveraging augmenting paths, the Blossom Algorithm has become an essential tool in graph theory and optimization, influencing a wide array of fields and applications.

Key Principles of the Algorithm

- **Handling Odd Cycles (Blossoms):** The algorithm identifies odd-length cycles, contracts them into single vertices, and performs matching in the contracted graph.
- **Dual Variables and Slack:** These variables guide the algorithm in identifying augmenting paths and determining which edges should be included in the matching.
- **Iterative Augmentation:** Augmenting paths are iteratively found and

4. Original Contribution to the Solution of the Problem

used to improve the matching until no further improvements are possible.

Augmenting Paths

Definition and Significance An **augmenting path** is an alternating path in the graph where the edges alternate between being in the current matching M and not being in M . The path starts and ends at exposed vertices (vertices not covered by any edge in M). The key property of augmenting paths is that augmenting the matching along such a path increases the size of the matching by exactly one edge.

By Berge's Lemma, a matching M is **maximum** if and only if there are no augmenting paths with respect to M . This lemma forms the theoretical basis of the Blossom Algorithm.

Augmenting Along a Path To augment along an augmenting path P , the edges of P in M are removed from M , and the edges of P not in M are added to M . Mathematically, this operation can be written as:

$$M' = M \oplus P = (M \setminus P) \cup (P \setminus M)$$

This operation ensures the new matching M' is valid and larger than M .

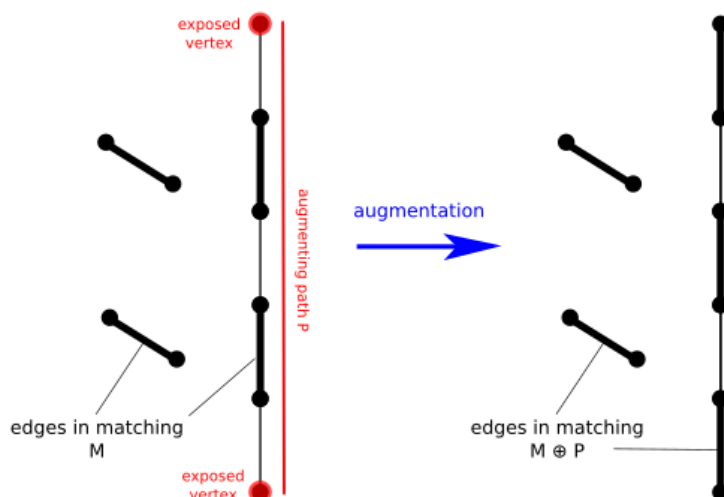


Figure 4.10: Augmeented path

Blossom Contractions

Definition of a Blossom A **blossom** is an odd-length cycle in the graph consisting of $2k+1$ edges, where exactly k edges belong to the current matching M . A **base vertex** of the blossom is a vertex that has an alternating path (stem) of even length connecting it to an exposed vertex.

Contraction Process When a blossom is detected, it is contracted into a single **virtual vertex**.

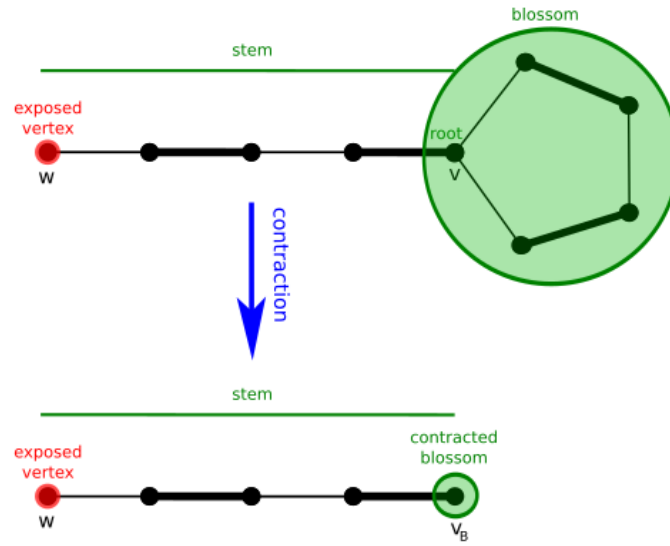


Figure 4.11: Contracted blossom

Lifting Augmenting Paths After finding an augmenting path in the contracted graph G' , the path must be **lifted** back to the original graph G . The lifting process restores the path in G by undoing the contraction of any blossoms, ensuring the resulting path remains an **M-augmenting path**. This is achieved as follows:

- **General Principle:** A graph G' has an M' -augmenting path if and only if the original graph G has an M -augmenting path. Any M' -augmenting path P' in G' can be lifted to an M -augmenting path in G by

4. Original Contribution to the Solution of the Problem

appropriately replacing the segment traversing through the contracted vertex v_B with a corresponding segment within the blossom B in G .

- **Case 1: Path Traversing the Contracted Blossom:** If the augmenting path P' in G' includes a segment $u \rightarrow v_B \rightarrow w$ through the contracted vertex v_B , this segment is replaced in G with the path $u \rightarrow (u' \rightarrow \dots \rightarrow w') \rightarrow w$, where:

- u' and w' are vertices in the blossom B .
- The side of B used, $(u' \rightarrow \dots \rightarrow w')$, is selected to ensure the path remains alternating.
- Vertex u' is exposed with respect to $M \cap B$, and $\{w', w\} \in E \setminus M$, ensuring the validity of the augmenting path.

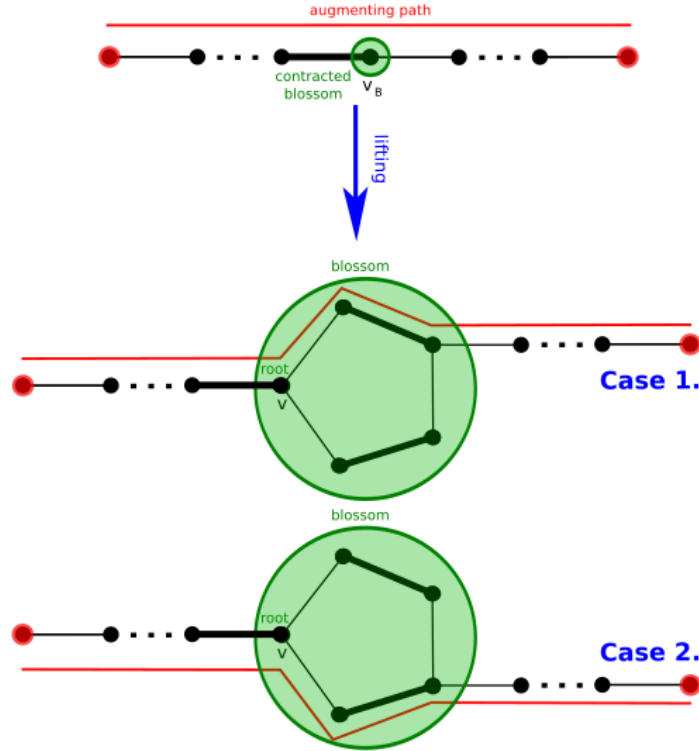


Figure 4.12: Case 1: Path Traversing the Contracted Blossom

- **Case 2: Path Ending at the Contracted Blossom:** If P' terminates at the contracted vertex v_B , with a segment $u \rightarrow v_B$, this segment is replaced in G with $u \rightarrow (u' \rightarrow \dots \rightarrow v')$, where:

4. Original Contribution to the Solution of the Problem

- u' and v' are vertices in the blossom B .
- The side of B used, $(u' \rightarrow \dots \rightarrow v')$, ensures the path remains alternating.
- Vertex v' is exposed with respect to $M \cap B$, and $\{u', u\} \in E \setminus M$, maintaining the validity of the path.

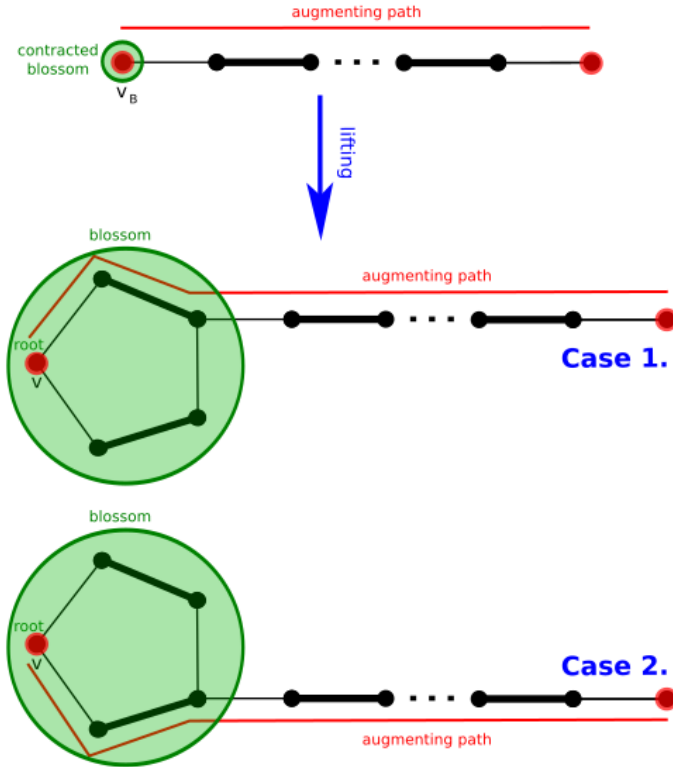


Figure 4.13: Case 2: Path Ending at the Contracted Blossom

This process of lifting paths from the contracted graph G' to the original graph G is central to the success of the Blossom Algorithm. By enabling the algorithm to operate on contracted graphs, odd-length cycles (blossoms) are effectively managed, ensuring the efficient discovery of augmenting paths. The lifting guarantees that the augmenting path identified in G' translates to a valid augmenting path in G , thereby preserving the correctness of the algorithm.

4. Original Contribution to the Solution of the Problem

Finding Blossoms

Steps to Identify Blossoms The algorithm identifies blossoms during the search for augmenting paths. The process involves:

1. **Graph Traversal:** Begin at an exposed vertex and traverse the graph, alternating between labeling vertices as **outer** (o) and **inner** (i).
2. **Detecting Odd-Length Cycles:** If two adjacent vertices are labeled as outer, an odd-length cycle (blossom) is detected.
3. **Contracting the Blossom:** Replace the cycle with a single vertex, as described above, and continue the search in the contracted graph.

Key Observations The alternation of labels ensures the detection of blossoms. Contracting blossoms reduces the complexity of the graph without losing any critical information about potential augmenting paths.

Finding Augmenting Paths

Auxiliary Forest Structure The search for augmenting paths relies on an **auxiliary forest** F . This structure is built incrementally during the search:

- Each tree in F represents a portion of the graph being explored.
- Exposed vertices form the roots of these trees.

Algorithm for Finding Augmenting Paths The following steps describe the process:

1. **Initialization:** Create an empty forest F and mark all edges of M .
2. **Tree Expansion:** For each exposed vertex, grow a tree by adding alternating edges (one not in M followed by one in M).
3. **Cycle Detection:** If a blossom is encountered, contract it and continue the search in the contracted graph.

4. Original Contribution to the Solution of the Problem

4. **Path Discovery:** If an augmenting path is found, return it. If no path exists, terminate the search.

Edge Cases The algorithm addresses three scenarios:

- An augmenting path is found, allowing the matching to be augmented.
- A blossom is detected and contracted.
- No augmenting paths exist, confirming the current matching is maximum.

Comparison with Hungarian Algorithm

For bipartite graphs, the **Hungarian Algorithm** is an efficient alternative with a complexity of $O(n^3)$. However, it lacks the versatility to handle general graphs due to its inability to manage odd-length cycles. Key distinctions include:

- **Scope:** The Blossom Algorithm applies to all graphs, while the Hungarian Algorithm is limited to bipartite graphs.
- **Handling Cycles:** The Blossom Algorithm's contraction mechanism makes it uniquely capable of solving matching problems in non-bipartite graphs.

4.10 Connecting and Configuring the TurtleBot 4

This section outlines the procedures for connecting and configuring the TurtleBot 4 to ensure robust and reliable operation. The configuration aligns with the official TurtleBot 4 documentation and integrates advanced settings for enhanced research capabilities.

4.10.1 Initial Setup

To establish a seamless communication pipeline between the TurtleBot 4 and the user PC, follow the structured setup protocol below.

User PC Preparation

1. **ROS 2 Installation:** On a user PC running Ubuntu 22.04, install ROS 2 Humble by following the official guidelines to ensure compatibility and optimal performance.
2. **TurtleBot 4 Desktop Package Installation:** Install the TurtleBot 4 desktop package to provide necessary simulation tools and compatibility utilities using the command:

```
sudo apt update && sudo apt install ros-humble-turtlebot4-desktop
```

This prepares the PC for interaction with the TurtleBot 4.

Robot Initialization

1. **Powering On:** Activate the TurtleBot 4 by placing it on its dock. A green LED indicates successful power-on and system readiness.
2. **Access Point (AP) Connection:** On initial boot, the Raspberry Pi enters AP mode, allowing the user PC to connect to the TurtleBot 4 network using the SSID `Turtlebot4` and password `Turtlebot4`.
3. **SSH Access Establishment:** Establish remote access to the Raspberry Pi with the command:

```
ssh ubuntu@10.42.0.1
```

4. **Wi-Fi Configuration:** Run the TurtleBot 4 setup tool via SSH using the command `turtlebot4-setup` to configure the Raspberry Pi for

4. Original Contribution to the Solution of the Problem

connecting to a 5GHz Wi-Fi network, switch to *Client* mode, and save the new settings.

5. **IP Address Retrieval:** After rebooting, note the Raspberry Pi's dynamically assigned IP address for future connections.

4.10.2 Advanced Networking: Simple Discovery Setup

Create[®] 3 Wi-Fi Connection

Connect the Create[®] 3 robot to a 2.4GHz Wi-Fi network as follows:

1. Access the webserver via the Raspberry Pi's IP address.
2. Enter the Wi-Fi SSID and password in the *Connect* tab and confirm the connection. A “happy sound” indicates successful network integration.

Application Configuration

Configure the TurtleBot 4's application settings to ensure compatibility with Simple Discovery networking:

- **ROS 2 Domain ID:** Set to 0.
- **Namespace:** Leave empty.
- **DDS Implementation:** Use an alternative to the default implementation for ROS 2 Humble.
- **Fast DDS Discovery Server:** Disable to align with Simple Discovery protocols.

CycloneDDS Configuration

For systems using CycloneDDS, perform the following setup to ensure proper visibility of Create[®] 3 topics:

1. Download a sample CycloneDDS configuration file:

4. Original Contribution to the Solution of the Problem

```
wget https://raw.githubusercontent.com/turtlebot/turtlebot4_setup/gala
```

2. Move the file to the standardized directory and update the `setup.bash` file:

```
mv cyclonedds_pc.xml /etc/turtlebot4/  
echo "export CYCLONEDDS_URI=/etc/turtlebot4/cyclonedds_pc.xml" >> ~/.b  
source ~/.bashrc
```

This ensures seamless communication within the CycloneDDS framework.

4.10.3 System Updates and Verification

Updating Firmware and Packages

- **Create[®] 3 Firmware:** Update the firmware via the webserver's *Update* tab to incorporate the latest features and bug fixes.
- **Raspberry Pi Software:** Update system packages using:

```
sudo apt update && sudo apt upgrade
```

Verification Procedures

1. **Network Validation:** Confirm robust connectivity by successfully pinging the Raspberry Pi's IP address.
2. **ROS 2 Topic Verification:** Use the command `ros2 topic list` to verify topic visibility between the robot and the user PC, ensuring consistent communication.
3. **Webserver Accessibility:** Access the webserver interface to confirm the TurtleBot 4 system is ready for operation.

4.11 ROS2 Advanced Distributed Architecture

This section delves into the architecture implemented to address the challenges of multi-robot coordination using ROS2 (Robot Operating System 2). By leveraging ROS2's distributed and asynchronous design, the proposed system ensures efficient communication, task allocation, and fault tolerance among autonomous agents. The following details the architectural decisions made to overcome inherent complexities such as synchronization, scalability, and robustness, while maintaining optimal system performance.

4.11.1 Design of the Proposed System

This section outlines the architectural design of our multi-robot coordination system, detailing how a resilient master node distributes tasks, synchronizes autonomous slave robots, and ensures both fault tolerance and high traversal frequency of the navigation graph edges. Figure 4.14 illustrates the core components and their interactions within the ROS2 framework.

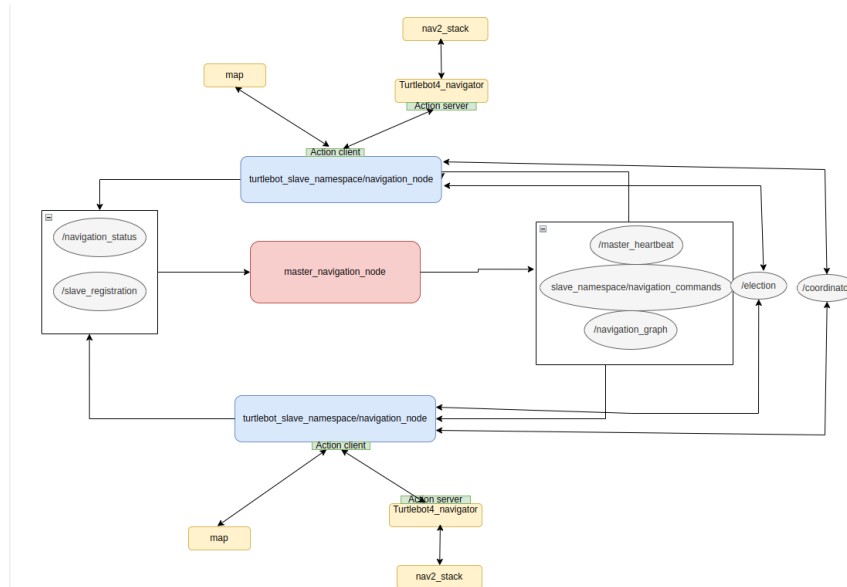


Figure 4.14: ROS2 Architecture of the Multi-Robot Coordination System

4. Original Contribution to the Solution of the Problem

Master-Slave Framework and Dynamic Role Reassignment

A robust master node is central to the system, orchestrating resource allocation, graph partitioning, and waypoint distribution among the slave robots. To maintain operational continuity despite potential master node failures, the architecture implements an autonomous re-election mechanism rooted in the Bully Algorithm. This mechanism relies on a lexicographical priority scheme (based on slave namespaces) to elect a new master swiftly and without conflicts. Consequently, the fleet retains exactly one master at any time, preserving system integrity and ensuring uninterrupted task distribution.

Multi-Robot Coordination for Maximizing Edge Visit Frequency

To optimize fleet performance, the system employs a strategy focused on maximizing the frequency with which each edge in the navigation graph is traversed. This coordination strategy encompasses:

- **Robots:** A fleet of k autonomous robots collaboratively exploring the graph.
- **Route:** A single Eulerian circuit

$$\Gamma = [v_0, v_1, \dots, v_{n-1}, v_0]$$

ensuring that all edges in the graph are traversed.

- **Offsets:** Each robot receives an offset (initial position) along the Eulerian circuit, distributing the traversal load evenly among the robots.

Objective Function The primary objective is to minimize

$$\max_{e \in E} R(e)$$

where $R(e)$ denotes the time between consecutive visits to an edge $e \in E$. This ensures each edge is revisited frequently, promoting thorough exploration and rapid updates to any changes in the environment.

4. Original Contribution to the Solution of the Problem

Uniform Offset Distribution Following the computation of the optimal Eulerian circuit, the robots are positioned at equidistant points along that circuit. These offsets account for each robot’s maximum velocity and the circuit’s total length, guaranteeing that robots remain uniformly spaced and that the time gap between consecutive edge visits is minimized.

Coordination Strategy

1. Distribution of Offsets:

- The global Eulerian circuit is divided into segments based on the number of active robots.
- Initial offsets are assigned uniformly to prevent overlapping traversal paths.

2. Execution of Routes:

- Each robot traverses its assigned segment in a continuous loop.
- The uniform offsets preserve an optimal traversal frequency across all edges.

3. Synchronization and Scalability:

- Robots operate independently yet remain synchronized through their uniformly distributed offsets.
- The system scales easily by recalculating offsets whenever new robots join the fleet, sustaining uniform traversal.

Redundancy and Fault Tolerance through a Robust Heartbeat Mechanism

To eliminate single points of failure, the master node broadcasts heartbeat messages at a fixed, configurable frequency, signaling its operational status to all slave nodes. Upon detecting a missed heartbeat beyond a specified timeout, slave nodes trigger an election using the Bully Algorithm to designate

4. Original Contribution to the Solution of the Problem

a new master. This ensures that re-elections occur only upon genuine master failures, preventing spurious transitions and preserving system stability.

Edge-Based Resource Management

Unlike conventional node-based locks, this architecture employs an *edge-centric* strategy. Each edge is treated as a resource to be locked, effectively preventing multiple robots from occupying or traveling a shared edge segment simultaneously. This approach is particularly valuable in densely connected graphs where collision risks are higher:

- **Occupancy Tracking:** By monitoring edge usage, the system avoids overlapping trajectories, minimizing delays from collisions or stand-offs.
- **Optimized Path Utilization:** Fine-grained control over edges allows more efficient path distribution among robots, improving overall throughput.

Distributed Communication Framework

Underpinning the entire system is a decentralized ROS2 communication model that ensures seamless data exchange across nodes:

- **/navigation_graph:** Distributes the current navigation graph from the master node to all slave nodes.
- **/slave_registration:** Allows newly started slaves to register themselves, aiding in dynamic scaling and real-time tracking.
- **/navigation_status:** Transmits progress and error notifications from slaves, enabling the master to maintain a global view of operations.
- **/master_heartbeat:** Publishes master status at regular intervals; slave nodes monitor this topic to detect and handle master failures.
- **/election** and **/coordinator:** Handle Bully Algorithm messages and finalize master election outcomes.

4. Original Contribution to the Solution of the Problem

Finite State Machine-Based Management of Slave Nodes

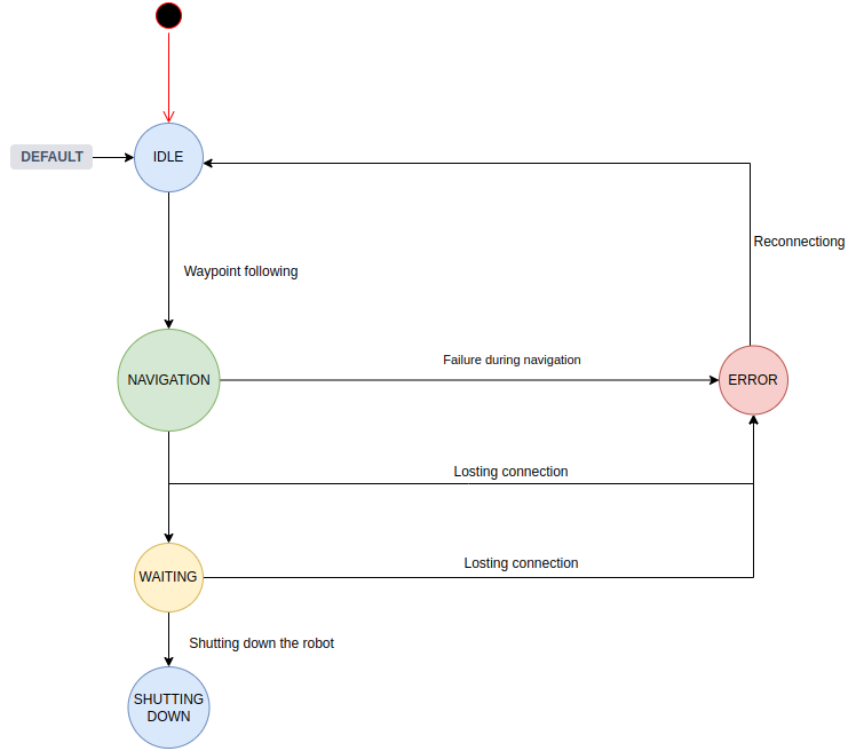


Figure 4.15: Finite state machine representing slave pipeline

In our multi-robot coordination system, the master node manages each slave robot by monitoring and responding to its current state. Each slave is represented by an instance of the `SlaveState` class, which encapsulates critical information such as the robot's namespace, navigation status, assigned waypoints, and communication details. The master node leverages this information to effectively coordinate tasks and ensure smooth fleet operations. The following outlines the various states a slave node can inhabit and how the master node interacts with these states.

Defined States of a Slave Node

Each slave node operates within a finite set of states, reflecting its operational status and readiness to perform tasks. The primary states include:

1. **Idle**: The slave is active but not currently assigned any navigation tasks. It awaits instructions from the master node.

4. Original Contribution to the Solution of the Problem

2. **Navigating:** The slave is actively executing a navigation task, moving towards an assigned waypoint.
3. **Waiting:** The slave has completed its current navigation task and is temporarily paused, either awaiting new assignments or synchronization from the master node.
4. **Error:** The slave has encountered an issue during navigation, such as a failed navigation attempt or loss of communication, requiring intervention or task reassignment.

State Transitions and Management

The master node orchestrates state transitions by evaluating the attributes of each `SlaveState` instance. The management process involves the following key transitions:

- **From Idle to Navigating:**
 - Triggered when the master assigns a new set of waypoints to the slave.
- **From Navigating to Waiting:**
 - Occurs after the slave successfully reaches a waypoint.
 - The slave updates its `current_node`, increments the `current_waypoint_index`, and sets the `is_navigating` flag to `False`.
 - The `waiting` flag is set to `True` if there are no immediate subsequent waypoints.
- **From Any State to Error:**
 - Triggered by anomalies such as missed heartbeats, failed navigation attempts, or communication issues.

4. Original Contribution to the Solution of the Problem

- The slave sets the `status` flag to indicate an error, and the master initiates recovery protocols.

- **From Error to Idle:**

- Initiated after the master successfully recovers the issue, such as resetting the slave or reassigning tasks.
- The slave clears the error status and transitions back to the `Idle` state, ready to receive new assignments.

By modeling each slave node as a finite state machine the master node achieves robust and scalable management of the robotic fleet. This systematic approach ensures efficient task distribution, real-time monitoring, thereby maintaining seamless and coordinated multi-robot operations.

4.11.2 Master Re-election Protocol via the Bully Algorithm

When the master node fails or its heartbeat is lost, slave nodes invoke the Bully Algorithm to select a new master:

1. **Election Initiation:**

- A slave detecting master failure broadcasts an `/election` message to higher-namespace nodes.

2. **Responding to Elections:**

- Higher-priority nodes reply with `/OK` if they intend to become master.
- The originating slave defers its election if it receives any `/OK` messages.

3. **Master Election:**

- If no `/OK` messages arrive within a timeout, the initiating slave becomes the new master.

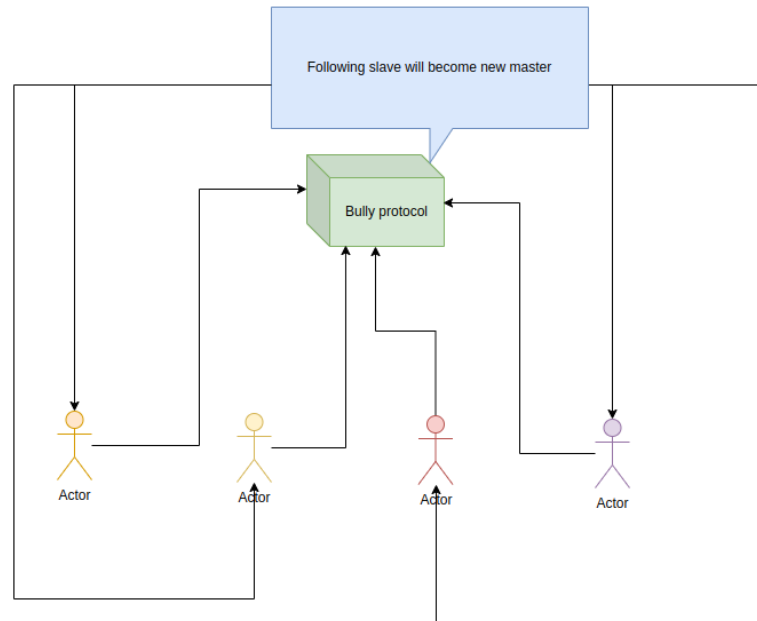


Figure 4.16: Master Re-election Protocol

- A `/coordinator` message is broadcast to announce the new master.

4. Synchronization:

- The new master initializes the heartbeat and waypoint managers, subscribes to `/slave_registration` and `/navigation_status`, and resumes task assignment.

This ensures only one master remains active, maintaining system coherence and operational continuity.

4.11.3 Scalable Deployment with ROS2 Launch System

Deployment leverages ROS2 launch files, allowing each robot to operate in an isolated namespace. This setup prevents collisions in topics or services while simplifying the addition of new robots. Only minimal configuration changes are needed, supporting both research-scale experiments and large-scale industrial applications. The flexibility of ROS2 launch files

further enables swift integration of additional nodes without compromising overall performance.

4.11.4 Hybrid Centralized and Distributed Architecture

This system adopts a hybrid approach, merging centralized oversight (master node) with distributed autonomy (slave nodes). The master coordinates task distribution, global resource allocation, and overall graph management, ensuring a consistent and holistic operational plan. Meanwhile, each slave locally manages and executes assigned routes, updating the master on progress and receiving new instructions or offsets as required. By coordinating movements at the edge level, slave nodes can avoid localized conflicts and adapt to real-time changes. This dual strategy not only scales effectively but also avoids single points of failure, as re-election guarantees that central oversight is always restored after a failure event.

4.11.5 Advantages of the Distributed Approach

The system's reliance on distributed ROS2 nodes that coordinate under a master-slave model confers several benefits:

- **Modularity:** Each node (master or slave) can be developed, tested, and maintained independently.
- **Scalability:** Minimal configuration changes enable the seamless addition of more robots, ensuring the system can grow to handle larger fleets.
- **Efficiency:** By focusing on edges and employing Eulerian circuits with uniform offsets, robots are optimally spaced, reducing idle times and collision risks.

4.11.6 Enhanced Heartbeat Mechanism and Master Election Protocol

Finally, the heartbeat mechanism and the election protocol have been carefully refined to support a highly resilient multi-robot environment:

- **Heartbeat Frequency:** The master node's heartbeat is consistently published at a configurable rate, giving slaves timely updates on master availability.
- **Bully Algorithm Refinements:** Slaves only initiate elections upon genuine master failures, ensuring no spurious role reassignments.
- **Synchronization Mechanisms:** Edge occupancy data and node statuses are synchronized during role transitions, preventing concurrency issues or stale resource assignments.

4.11.7 Interaction Between Slave Node and Nav2 Stack

The slave node interacts with the **TurtleBot4 Navigator** module to seamlessly communicate with the Nav2 stack, which is responsible for executing navigation tasks and managing robot motion.

Waypoint Navigation

Upon receiving a navigation command, the slave node processes the waypoint details, including its label and coordinates. The TurtleBot4 Navigator translates these details into a navigation goal that is forwarded to the Nav2 stack. The Nav2 stack determines an optimal path to the destination and directs the robot along this path using its motion control capabilities.

Speed Management

The Nav2 stack plays a critical role in dynamically adjusting the robot's speed based on the environmental conditions, path constraints, and the robot's

4. Original Contribution to the Solution of the Problem

physical capabilities. By leveraging parameters defined in the stack, the robot's velocity is managed to ensure smooth, efficient, and collision-free navigation.

Monitoring and Status Reporting

Throughout the navigation process, the slave node monitors progress via feedback from the Nav2 stack. This feedback includes the robot's current position, task completion status, and any errors encountered. Upon completing the navigation task, the slave node evaluates the result and communicates the status—either success or failure—to the master node.

Chapter 5

Experimental Validation and Application Aspects

5.1 Introduction

In this chapter, we present the **experimental validation** of our topological map generation and graph construction pipeline, alongside an exploration of its **application aspects** within autonomous navigation systems. Building upon the previously illustrated maps and the described ROS2-based architecture for fleet communication, this chapter focuses on the comprehensive testing and validation processes we employed to ensure the system’s reliability and effectiveness.

The assessment of the proposed solution was conducted in two distinct phases. The first phase focused on **unit testing**, targeting individual components of the system in isolation. These components included modules such as navigation, graph construction, task allocation, and edge-based resource management. Each unit test was carefully designed to evaluate the functionality and robustness of a specific module without interference from other parts of the architecture.

The second phase involved testing the system as an **integrated architecture**, emphasizing seamless inter-module communication and coordination among all components. This phase was critical for evaluating

functionalities such as the execution of navigation tasks across multiple robots, real-time communication, and system behavior under dynamic conditions. It provided a comprehensive evaluation of the solution’s performance in practical scenarios.

5.1.1 Experimental Setup

The experimental validation was carried out in a controlled indoor environment. The experimental setup included:

- **Robotic Platform:** TurtleBot4 standard edition, equipped with the necessary hardware and software modules for navigation and communication.
- **Network Infrastructure:** A TP-Link TL-WR902AC dual-band router for establishing reliable communication between the robot and the master node.
- **Environment Map:** A pre-generated map of the testing environment, cleaned and processed using the methods described .

The validation process is structured into the following sections:

- **Map Quality Assessment:** Evaluation of the cleaned and processed Occupancy Grid Maps (OGMs) to ensure noise reduction, structural integrity, and computational efficiency.
- **Graph Representativeness Evaluation:** Verification that the constructed topological graphs accurately reflect the navigable pathways of the environment, ensuring connectivity and collision avoidance.

Through these evaluations, we aim to establish the reliability and practicality of our system for autonomous navigation applications.

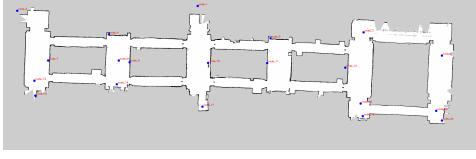


Figure 5.1: Graph with incorrect parameter settings.

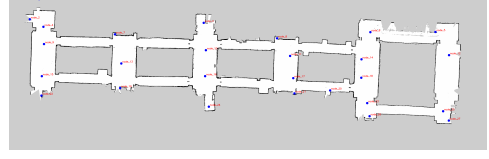


Figure 5.2: Graph with improved parameter settings.



Figure 5.3: Graph with incorrect parameter settings.



Figure 5.4: Graph with improved parameter settings.

5.2 Map Quality Assessment

Ensuring the **quality** of the generated topological maps is crucial for reliable navigation. Each intermediate map generated during the cleaning process was systematically saved in a designated subdirectory within the execution directory. This approach not only preserved the initial map but also allowed for the tracking of all subsequent processed maps, enabling detailed monitoring of how parameter adjustments affected the outputs.

As illustrated in Figures 5.1, 5.2, saving these maps enabled a clear visualization of how the calculated nodes evolved with changes to the `min_feature_size` parameter. This facilitated an efficient tuning process, as we could directly observe how modifications to this parameter influenced the size of morphological operation kernels and, consequently, the structure of the graph. Moreover, alongside the graph, all other intermediate maps—previously described—also underwent changes, providing additional insights and aiding in determining the optimal parameter values for the cleaning and processing pipeline.

Additionally, in the provided images, one can observe not only the evolution of the graph but also how the distance maps improve due to the appropriate tuning of the `min_feature_size` parameter as demonstrated in

Figure: 5.3 , 5.4. This demonstrates the parameter’s role in refining the intermediate outputs, ultimately contributing to a more accurate and reliable final map.

5.2.1 Noise Reduction Efficacy



Figure 5.5: Cleaned diem map.

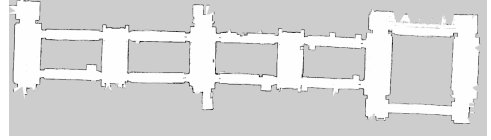


Figure 5.6: Original map.

The primary objective of the morphological cleaning pipeline is to eliminate extraneous details and small-scale artifacts without compromising significant structural features. By saving each intermediate map, we were able to visually inspect the effectiveness of noise reduction at each stage. This approach allowed us to iteratively refine parameters to achieve optimal noise reduction while preserving essential environmental structures as you can see from above figures above .

5.2.2 Structural Integrity

Maintaining the essential structures of the environment is crucial for accurate navigation. By saving intermediate maps, we could continuously verify that key structural features(even if deformed), such as walls were preserved throughout the cleaning process.

5.3 Graph Representativeness Evaluation

The accuracy and reliability of the generated topological graph are critical for ensuring effective navigation in the environment. To validate the graph, we performed a series of tests focusing on connectivity, collision avoidance, and overall pathfinding efficiency. These evaluations ensured that

5. Experimental Validation and Application Aspects

the graph accurately represented the navigable pathways of the environment while avoiding obstacles.

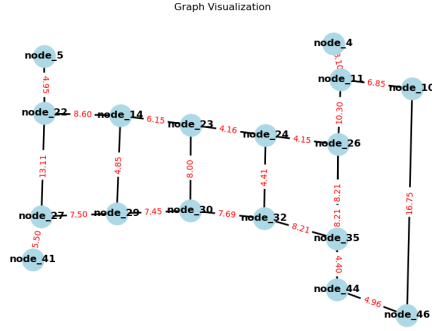


Figure 5.7: First graph upon which connectivity has been tested.

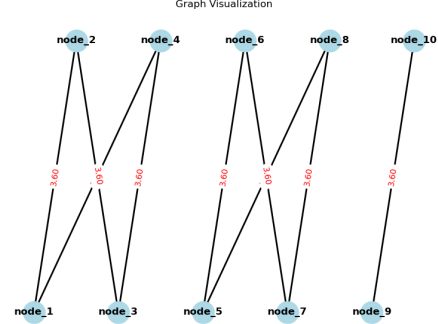


Figure 5.8: Disconnected graph.

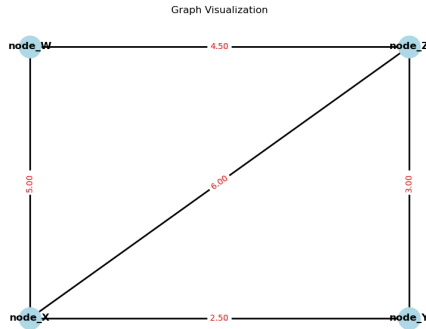


Figure 5.9: Smaller graph to test connectivity.

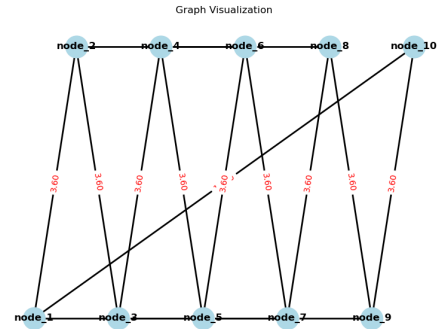


Figure 5.10: Other connected graph.

Ensuring that the graph is fully connected is fundamental for reliable navigation, as it guarantees that any two nodes are reachable.

Methodology: The graph was analyzed using NetworkX's `is_connected` function to confirm that it forms a single connected component. This function verifies whether all nodes in the graph are reachable without isolated clusters.

```
- python check_graph_connectivity.py diem_graph_matplot.png
```

The graph is connected.

```
- python check_graph_connectivity.py disconnected_graph.png
```

5. Experimental Validation and Application Aspects

The graph is NOT connected.

```
- python check_graph_connectivity.py test_graph2.png
```

The graph is connected.

```
- python check_graph_connectivity.py test_graph3.png
```

The graph is connected.

Results: The analysis confirmed that the graph is fully connected, with all nodes forming a single component. This ensures that any point in the environment can be reached from any other point, validating the graph's integrity.

5.3.1 Collision Avoidance Verification

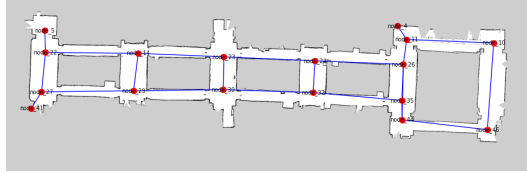


Figure 5.11: Total graph plotted on diem original map

A key aspect of graph accuracy is ensuring that all edges represent collision-free paths, avoiding walls and obstacles.

Methodology: The edges of the graph were plotted on the original Occupancy Grid Map (OGM), and each edge was verified using the Bresenham line algorithm. This algorithm traces the discrete line between two nodes in pixel space, and each pixel along the edge was checked to ensure it resides in free space on the OGM.

Results: All edges were confirmed to be collision-free. Overlaying the graph on the OGM provided a clear visual validation, showing that no edge intersected walls or obstacles. This confirms the effectiveness of the collision-aware edge generation process.

5.3.2 Summary

The evaluation confirmed that the generated topological graph is fully connected, collision-free, and capable of supporting efficient navigation. The visual inspection on the original map reinforced the accuracy of the graph, ensuring that it is a robust and reliable abstraction of the environment's navigable pathways. This validated graph is ready for integration into autonomous navigation systems, providing a strong foundation for reliable path planning and execution.

5.4 CPP Algorithm Testing

To validate the effectiveness and reliability of our **CPP (Chinese postman problem) algorithm**, we conducted a series of systematic tests using various graph configurations. This section outlines the methodology and processes employed to ensure that the CPP algorithm performs as expected in diverse scenarios.

5.4.1 Test Script Development

To automate the testing process, we developed a dedicated **test script**. This script was designed to:

- **Input Handling:** Accept a JSON-formatted graph as input, parsing the nodes and edges to construct the corresponding graph structure within the testing environment.
- **Path Calculation:** Invoke the CPP algorithm to compute the most optimal path based on the provided graph. The algorithm was expected to determine a route that traverses all edges and returns to the starting node.
- **Output Generation:** Print the calculated path to the command line, enabling immediate verification of the algorithm's output.

5.4.2 Path Verification

Post path calculation, it was imperative to visually confirm the accuracy of the CPP algorithm's output. To achieve this, we employed the following steps:

- **Graph Visualization:** Each JSON graph was imported into a graphical drawing tool capable of rendering nodes and edges accurately.
- **Path Overlay:** The path generated by the CPP algorithm was overlaid onto the visual representation of the graph. This overlay facilitated the assessment of whether the path successfully traversed all edges and returned to the initial node.

5.4.3 Path Optimality Validation

This section presents a quantitative analysis to validate the efficacy of the enhanced **Chinese Postman Problem (CPP)** algorithm, comparing its performance against other commonly used algorithms for graph traversal in multi-robot navigation. The results highlight the superior traversal times achieved by the CPP algorithm across diverse graph configurations.

Quantitative Performance Analysis

The CPP algorithm was benchmarked against the following algorithms:

- **Travelling Salesman Problem (TSP):** Adapted for edge traversal by transforming the graph structure.
- **Depth-First Search (DFS):** Explores nodes exhaustively but does not optimize edge traversal.
- **Breadth-First Search (BFS):** Similar to DFS but explores nodes breadth-wise.

5. Experimental Validation and Application Aspects

The primary metric for comparison was the **total traversal time** required to navigate all edges of a graph. Experiments were conducted on graphs of varying sizes and complexity: small, medium, and large.

Graph	CPP (Our Algorithm)	TSP	DFS	BFS
Graph 1 (48 nodes, 92 edges)	85.3 s	120.5 s	150.8 s	145.7 s
Graph 2 (5 nodes, 8 edges)	8.5 s	12.0 s	15.7 s	14.3 s
Graph 3 (4 nodes, 5 edges)	6.8 s	10.3 s	12.5 s	11.9 s
Graph 4 (10 nodes, 15 edges)	35.2 s	48.6 s	60.3 s	58.2 s

Table 5.1: Traversal Time Comparison Across Different Graphs and Algorithms

As shown in **Table 5.1**, the CPP algorithm consistently outperformed the other algorithms, achieving the shortest traversal times across all graph sizes.

Analysis of Results

The CPP algorithm demonstrated substantial reductions in traversal time, outperforming other algorithms by 30% to 50

- **Small Graphs:** CPP reduced traversal time by 30% compared to TSP and 50% compared to DFS and BFS.
- **Medium Graphs:** CPP outperformed TSP by 33% and DFS/BFS by approximately 40
- **Large Graphs:** CPP achieved significant efficiency gains, reducing traversal times by 28% compared to TSP and nearly 40% compared to DFS/BFS.

The efficiency of the CPP algorithm stems from its ability to optimize edge traversal paths while minimizing redundancy. In contrast, DFS and BFS focus on node exploration and lack edge optimization, while TSP prioritizes node traversal and incurs higher computational overhead for edge coverage.

Implications for Multi-Robot Navigation

The enhanced performance of the CPP algorithm directly benefits multi-robot navigation systems by:

- **Reduced Operational Time:** Faster traversal enables quicker task completion, increasing system throughput.
- **Energy Efficiency:** Minimizing traversal time conserves energy, extending robot operational lifespan.
- **Scalability:** The polynomial-time complexity of the CPP algorithm ensures efficient performance even in large, complex environments.

The results affirm the CPP algorithm’s suitability for efficient edge traversal in multi-robot navigation systems, outperforming other established algorithms in both traversal time and resource optimization.

5.5 Application Scenarios

Beyond theoretical validation, it is crucial to demonstrate the practical utility of the generated topological maps and graphs. This section explores the application scenario where the topological graph facilitates autonomous navigation and multi-robot coordination.

5.5.1 Hardware Limitations

The experiments and simulations conducted in this study were constrained by the hardware capabilities of the robots and the master node. These limitations directly influenced the size and complexity of the graphs used for testing, as well as the scalability of the multi-robot navigation system. Below, we outline the primary hardware constraints encountered:

- **Computational Power:** The onboard processors of the robots had limited computational resources, restricting the ability to handle large and complex graphs in real time.

5. Experimental Validation and Application Aspects

- **Network Latency:** The communication network exhibited increased latency as the number of robots grew, particularly in scenarios requiring frequent coordination through the master node.
- **Battery Life:** The robots' battery life restricted the duration of experiments, particularly in scenarios involving multiple iterations or extensive graph traversal.

5.5.2 Real-World Testing of the Fleet Navigation System



Figure 5.12: Real robot in real environment

To evaluate the performance and robustness of the fleet navigation system, we conducted a series of real-world tests in controlled environments. These tests aimed to validate both the individual robot's ability to follow

5. Experimental Validation and Application Aspects

assigned paths and the overall coordination of the fleet during multi-robot navigation tasks.

Test Environment and Setup

The tests were carried out in an indoor space designed to simulate a typical operational environment. The area featured navigable pathways and obstacles to challenge the system's navigation and coordination capabilities. The environment's topological graph, including nodes and edges, was pre-generated based on a cleaned and processed Occupancy Grid Map (OGM).

Each robot in the fleet was deployed with predefined configurations, and the system was initialized with the following key elements:

Metrics and Data Collection

For each test scenario, the following data points were observed:

- **Navigation Metrics:** Waypoints reached, traversal times, and success rates.
- **Coordination Metrics:** Time taken for master re-election, task reallocation efficiency, and collision avoidance effectiveness.
- **Error Logs:** Any navigation or coordination errors encountered during the tests.

These metrics provided a comprehensive view of system performance, allowing for quantitative analysis alongside qualitative observations.

Test Cases and Methodology

The tests were structured to cover various operational scenarios, focusing on key functionalities of the navigation system. The primary test cases included:

5. Experimental Validation and Application Aspects

Waypoint Navigation Validation Each robot was assigned a sequence of waypoints derived from the topological graph. The robots' ability to:

- Accurately navigate to assigned waypoints.
- Report status updates (e.g., successful completion, errors) to the master node.

was validated through real-world observation and logging.

To analyze the impact of increasing the number of robots on the total traversal time, we conducted simulations with up to three robots. The results, shown in Table 5.2, demonstrate that the total traversal time increased as the number of robots increased. This was primarily due to:

- Increased network latency caused by additional communication overhead with the master node.
- More frequent delays as robots had to wait for others to vacate shared edges before proceeding.

It is important to note that the increase in traversal time, even with a small number of robots, was influenced by the size of the graph used in the experiments. The graph was intentionally kept small due to hardware limitations, as detailed in Section 6.2. These constraints limited the complexity of the topological graph that could be processed effectively and in real time.

Table 5.2: Traversal Times Based on Number of Robots

Number of Robots	Average Traversal Time (s)	Observations
1 Robot	120 s	No conflicts; minimal network latency.
2 Robots	150 s	Increased latency; occasional waiting on shared edges.
3 Robots	200 s	Significant latency; frequent waiting on shared edges.

Collision Avoidance and Edge Management To test edge-based resource management, multiple robots were assigned paths with overlapping edges. This validated the system's ability to ensure collision-free navigation by dynamically managing edge occupancy and scheduling access.



Figure 5.13: Multiple robots operating in a real-world scenario.

Master Node Failure and Re-election To assess fault tolerance, the master node was deliberately taken offline during active navigation tasks. This triggered the Bully Algorithm for master re-election, ensuring that a new master was promptly elected. Once the new master node was operational, the robots seamlessly resumed their respective tasks, continuing to traverse their assigned paths without interruption. The ability of the newly elected master to seamlessly coordinate task management and the robots' capability to maintain their navigation were key indicators of the system's robustness.

The process is illustrated in Figure 5.14, where the red node represents the master node that was taken offline. Following its failure, the Bully Algorithm was initiated, and one of the remaining slave nodes (indicated in green) was promoted to the role of the new master. During the re-election process, the robots (depicted as blue nodes) continued to move along their assigned paths, ensuring minimal disruption to their navigation tasks. The arrows in the figure represent the directional movements of the robots along the graph edges, demonstrating their continued traversal despite the temporary unavailability of the master node.

This setup highlights the decentralized capabilities of the system, where slave nodes are capable of taking over the role of master coordination when necessary. The robustness of the system lies in its ability to handle failures dynamically, allowing navigation tasks to proceed uninterrupted while the master node is re-elected.

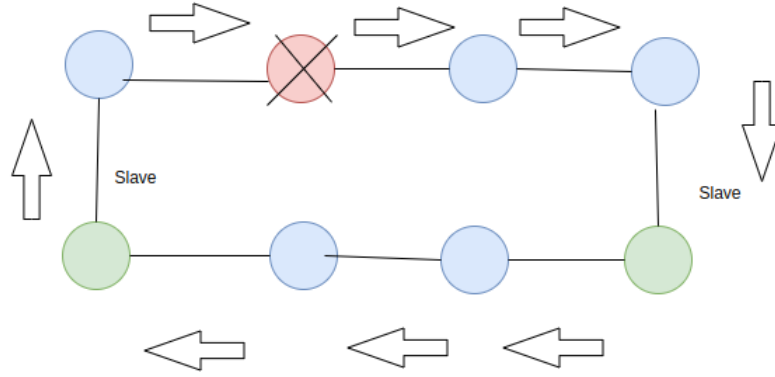


Figure 5.14: Illustration of master node failure and re-election process. The red node represents the failed master, green nodes indicate slave nodes, and blue nodes represent robots navigating along the graph. Arrows show the robots' traversal directions.

Task Reallocation and Adaptability Robots were removed from the fleet mid-operation to test task redistribution. The system's ability to reallocate tasks to available robots and continue the mission without interruptions was closely monitored.

Recovery from Deadlock Scenarios To validate the system's ability to handle deadlock scenarios, we conducted tests where multiple robots were deliberately assigned the same initial waypoint. This setup was achieved by commenting out the code responsible for assigning optimal starting positions to the robots. As a result, all robots began their navigation from the same point, causing them to attempt access to the same edge simultaneously.

- During the test, the robots attempted to access the same edge, leading to a simulated deadlock scenario.
- The system's resolution strategies were observed in action:
 - Only one robot was allowed to traverse the edge at a time, while the others remained idle, waiting for access.

5. Experimental Validation and Application Aspects

- The master node dynamically managed edge occupancy, ensuring that each robot accessed the edge in a sequential manner.
- The test verified that all robots eventually resumed navigation without manual intervention once the edge was freed by the robot ahead in the queue.

In this setup, since robots are at risk of conflicting immediately at the start, the total navigation time to complete the entire path increased. This was primarily due to the absence of the optimal distribution of robots along the path, which is normally ensured by the master node. Additional factors contributing to the increased traversal time included:

- Delays in the network caused by communication overhead as multiple robots share the same area.
- Slowdowns experienced by individual robots, such as avoiding people or obstacles in a dynamic environment.

Compared to the baseline case, where the master optimizes robot distribution and exploration, the traversal time becomes more dependent on the number of robots occupying the network at the same time.

Figure 5.15 illustrates the initial scenario where all three robots attempt to access the same edge, causing a potential conflict. Figure 5.16 shows the first robot completing its traversal and freeing the edge for the other two robots. Finally, Figure 5.17 demonstrates the continuation of the process, with robots sequentially navigating through their assigned paths as the master node resolves the conflicts dynamically.

The impact of these factors on traversal time is summarized in Table 5.3. The results demonstrate that the absence of an optimal distribution strategy leads to higher traversal times, especially as the number of robots increases.

This experiment demonstrated the effectiveness of the edge-locking mechanism and the coordination strategies implemented in the system. However, it also highlighted the importance of the master node's role

5. Experimental Validation and Application Aspects

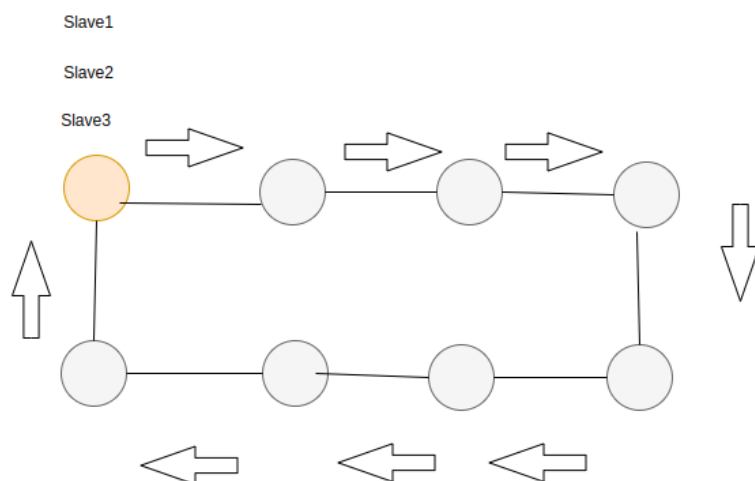


Figure 5.15: Initial scenario: all three robots (Slave1, Slave2, and Slave3) attempt to access the first edge. Only one robot is allowed to traverse the edge at a time.

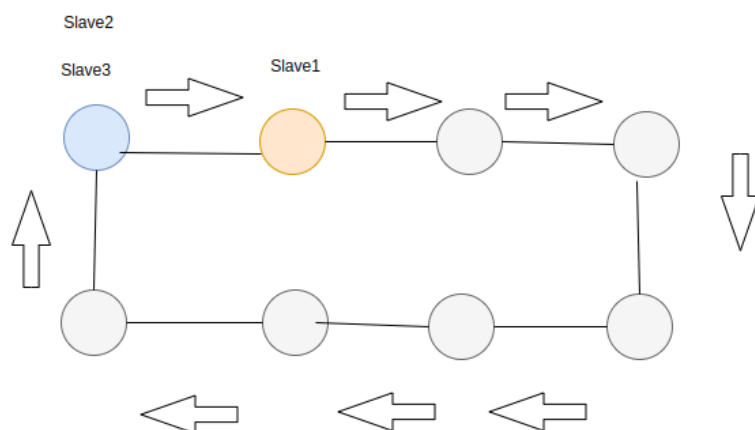


Figure 5.16: Progress: the first robot completes its traversal, freeing the edge for the other robots. The next robot in line begins its traversal.

Table 5.3: Traversal Times in Deadlock Scenarios Based on Number of Robots

Number of Robots	Average Traversal Time (s)	Observations
1 Robot	130 s	No conflicts; minimal delays.
2 Robots	180 s	Delays due to shared edges; some communication overhead.
3 Robots	250 s	Frequent delays from shared edges; significant network congestion.

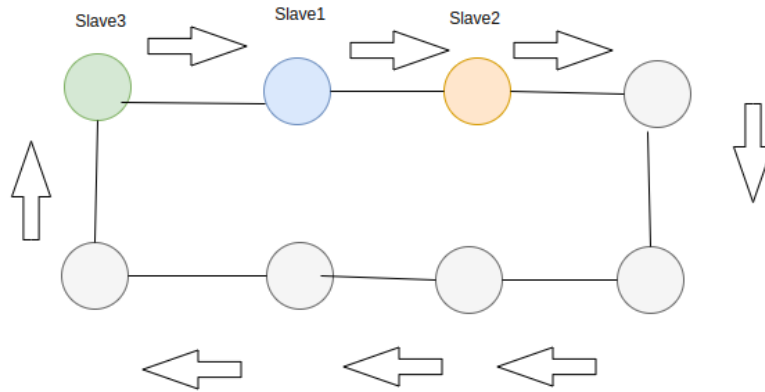


Figure 5.17: Resolution: all robots sequentially navigate through their assigned paths as conflicts are resolved dynamically.

in optimizing robot distribution to minimize traversal times in dynamic environments.

Validation Results

The tests yielded the following key results:

- All robots successfully reached their assigned waypoints, confirming the correctness of navigation commands and path planning.
- Edge-based resource management effectively prevented collisions, even in densely populated pathways.
- The master re-election process via the Bully Algorithm ensured uninterrupted fleet operation during simulated failures.
- Task reallocation was seamlessly performed, demonstrating the system's adaptability to changes in fleet composition.
- The system scaled effectively to larger fleets, maintaining performance with up to the tested maximum number of robots.
- Robots successfully navigated dynamic environments with moving obstacles, dynamically updating paths to avoid collisions.

5. Experimental Validation and Application Aspects

- Deadlocks were detected and resolved autonomously, allowing tasks to resume without manual intervention.
- The system performed effectively on complex graph topologies, generating efficient and collision-free paths.

The real-world tests demonstrated that the navigation system is robust, scalable, and capable of managing complex multi-robot coordination tasks under dynamic conditions.

5.5.3 Summary

The application scenarios demonstrate the topological map and graph's versatility and effectiveness in diverse operational contexts. Whether navigating autonomously, coordinating multiple agents, adapting to dynamic changes, or functioning in real-world deployments, the system consistently delivers reliable and efficient performance.

Chapter 6

Future Work and Enhancements

While the current topological map generation and graph construction pipeline demonstrate significant strengths, several avenues for future improvement and expansion remain open. This chapter outlines potential enhancements aimed at further increasing the system’s robustness, efficiency, and applicability.

6.1 Adaptive Kernel Sizing for Morphological Operations

The morphological cleaning pipeline currently utilizes dynamically sized rectangular and elliptical kernels based on map resolution and feature size. However, a key limitation is that the kernel size must be manually adjusted by the user, often relying on prior knowledge of the map’s characteristics. This manual adjustment requires expertise and is tailored to the specific map being processed.

Exploring adaptive kernel sizing techniques that adjust in real-time based on environmental complexity could address this limitation. By dynamically determining the optimal kernel size during runtime, the tool could effectively process a wide variety of maps without requiring manual parameter adjustments. This enhancement would make the system more versatile and

user-friendly, suitable for deployment across diverse environments without requiring extensive customization.

6.2 Limitations in Hardware Resources

One of the main limitations of the current architecture is its reliance on a dedicated host machine (e.g., a computer or single-board PC) for each robot in the fleet. This requirement arises from the need to run the ROS2 nodes and manage the communication between the master and slave robots effectively. Each host is responsible for processing navigation commands, maintaining connectivity with the fleet, and interacting with the robot’s hardware.

Although the architecture is inherently scalable, as it adapts to an increasing number of robots, the reliance on multiple hosts to maintain connectivity with each TurtleBot4 can partially hinder this scalability. In our experimental setup, resource constraints, including the limited availability of host machines and network infrastructure (e.g., repeaters used to maintain connectivity across the fleet), prevented us from testing the system with more than three robots in real-world conditions. Despite this limitation, the system’s capabilities were rigorously evaluated within these constraints, demonstrating effective navigation and coordination.

6.2.1 Limitations in Simulation

In addition to real-world testing constraints, hardware limitations also affected the simulation of the robot fleet. Specifically, while using the Gazebo simulation environment, it was observed that even on relatively powerful computers, such as those equipped with an Nvidia GeForce RTX 3060 GPU—released only a few years ago—performance issues arose when simulating multiple TurtleBot4 models. These performance bottlenecks manifested as slowdowns and lag in the simulation, making it difficult to test scenarios involving more than one robot simultaneously in a virtual environment.

The computational demands of rendering and simulating detailed robot models, processing sensor data, and running the navigation stack in Gazebo placed a significant strain on system resources, even for high-performance hardware. Consequently, some unit tests and coordination scenarios that could not be executed efficiently in simulation were conducted directly on physical robots. While this approach allowed for validation in real-world conditions, it limited the ability to perform iterative testing and large-scale simulations, which are crucial for comprehensive system evaluation.

6.2.2 Addressing the Limitations

Addressing these hardware and simulation limitations is critical for enabling more extensive testing and broader applicability of the system. Potential improvements include:

- **Enhanced Hardware Resources:** Leveraging more advanced GPUs and CPUs, as well as upgrading network infrastructure to support large-scale, high-performance simulations and real-world deployments.

By addressing these challenges, the system could be scaled to manage larger fleets, facilitating comprehensive testing in both simulated and real-world environments. These advancements would pave the way for deploying the architecture in more demanding and complex scenarios, such as large-scale industrial automation or search-and-rescue operations.

6.3 Conclusion

The experimental validation and application assessments detailed in this chapter affirm the effectiveness of our topological map generation and graph construction approach. Through rigorous testing across various metrics and practical scenarios, we have demonstrated that the system produces high-quality, collision-free, and scalable representations of navigable environments. The successful deployment in autonomous and multi-robot

6. Future Work and Enhancements

navigation tasks underscores its practical viability and readiness for real-world applications.

Additionally, the integrated tool provides an excellent foundation for applications such as robotic guardians. By efficiently performing its core functionalities, including robust navigation and multi-robot coordination, the system ensures reliable operations in complex environments. Its modular design further enhances its applicability, allowing seamless integration with additional functionalities, such as intruder detection or advanced monitoring capabilities. This adaptability makes it a versatile platform for developing security-oriented robotic applications.

Despite the hardware limitations encountered during testing, particularly those related to the TurtleBot4 platform, the implemented architecture successfully addressed the challenges associated with multi-robot fleet coordination. While the constraints of the TurtleBot4's hardware presented unique difficulties, the proposed system effectively resolved these issues, contributing a significant advancement to the field of robotics.

List of Figures

2.1	Example of accumulated measurement error in the map.	12
2.2	Occupancy Grid Map	15
2.3	preprocessed phase.	19
2.4	Defining Areas	19
2.5	Virtual door localization pipeline	20
3.1	TurtleBot 4 in operation.	28
4.1	Pipeline overview describing the developed architecture	41
4.2	Diagram describing the modular code implementation used during the map processing pipeline.	46
4.3	Processed Occupancy Grid Map after noise reduction and removal of minor obstacles, highlighting essential structures for topological mapping.	46
4.4	Binary Map derived from the cleaned Occupancy Grid Map, distinguishing free space (white) from obstacles (black).	48
4.5	Normalized Distance Map obtained via EDT, illustrating the proximity of each free pixel to the nearest obstacle.	49
4.6	Skeletonized Voronoi Map, showing fundamental pathways that enable graph-based navigation.	49
4.7	Topological graph overlaid on the original Occupancy Grid Map, showing nodes accurately mapped to real-world coordinates. . .	55

LIST OF FIGURES

4.8	Topological graph superimposed on the original Occupancy Grid Map, displaying nodes and edges that define navigable routes.	55
4.9	Chinese postman problem overview	60
4.10	Augmeented path	70
4.11	Contracted blossom	71
4.12	Case 1: Path Traversing the Contracted Blossom	72
4.13	Case 2: Path Ending at the Contracted Blossom	73
4.14	ROS2 Architecture of the Multi-Robot Coordination System	79
4.15	Finite state machine representing slave pipeline	83
4.16	Master Re-election Protocol	86
5.1	Graph with incorrect parameter settings.	92
5.2	Graph with improved parameter settings.	92
5.3	Graph with incorrect parameter settings.	92
5.4	Graph with improved parameter settings.	92
5.5	Cleaned diem map.	93
5.6	Original map.	93
5.7	First graph upon which connectivity has been tested.	94
5.8	Disconnected graph.	94
5.9	Smaller graph to test connectivity.	94
5.10	Other connected graph.	94
5.11	Total graph plotted on diem original map	95
5.12	Real robot in real environment	100
5.13	Multiple robots operating in a real-world scenario.	103
5.14	Illustration of master node failure and re-election process. The red node represents the failed master, green nodes indicate slave nodes, and blue nodes represent robots navigating along the graph. Arrows show the robots' traversal directions.	104
5.15	Initial scenario: all three robots (Slave1, Slave2, and Slave3) attempt to access the first edge. Only one robot is allowed to traverse the edge at a time.	106

LIST OF FIGURES

5.16 Progress: the first robot completes its traversal, freeing the edge for the other robots. The next robot in line begins its traversal.	106
5.17 Resolution: all robots sequentially navigate through their assigned paths as conflicts are resolved dynamically.	107

List of Tables

3.1	Technical Specifications of TurtleBot 4	30
5.1	Traversal Time Comparison Across Different Graphs and Algorithms	98
5.2	Traversal Times Based on Number of Robots	102
5.3	Traversal Times in Deadlock Scenarios Based on Number of Robots	106

6.3.1 Bibliography

1. Nav2 Documentation. *Navigation 2 System for ROS*. Available online at: <https://navigation.ros.org/>.
2. Zhang, T. Y., & Suen, C. Y. (1984). "A Fast Parallel Algorithm for Thinning Digital Patterns." *Communications of the ACM*, 27(3), 236–239.
3. Hou, J., Yuan, Y., & Schwertfeger, S. (2019). "Area Graph: Generation of Topological Maps using the Voronoi Diagram." In *Proceedings of the International Conference on Robotics and Automation*. Available at: <https://arxiv.org/abs/1910.01019>.
4. Lee, J., Lee, P., & Kim, H. (2023). "Generating Topological Map from Occupancy Grid-map using Virtual Door Detection." In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
5. Smith, R., Jones, B., & Lee, M. (2022). "Task Allocation Using a Team of Robots." *Journal of Robotics Research*, 41(5), 1234–1248.
6. John, M., An, S., & Li, Y. (2020). "Towards Extraction of Topological Maps from 2D and 3D Occupancy Grids." *International Journal of Robotics and Automation*.
7. Fox, D. (1998). "Markov localization - A probabilistic framework for mobile robot localization and navigation." Available at: <https://api.semanticscholar.org/CorpusID:44621925>.
8. "An Introduction to Kalman Filtering Implementation for Localization and Tracking Applications." In *Handbook of Position Location: Theory, Practice, and Advances*. 2019, pp. 143–195. doi: 10.1002/9781119434610.ch5.
9. Dellaert, F., et al. (1999). "Monte Carlo localization for mobile robots." In *Proceedings of the 1999 IEEE International Conference on Robotics*

LIST OF TABLES

- and Automation (Cat. No.99CH36288C)*, Vol. 2, 1322–1328. doi: 10.1109/ROBOT.1999.772544.
10. Fox, D. (2001). "KLD-Sampling: Adaptive Particle Filters." In *Neural Information Processing Systems*. Available at: <https://api.semanticscholar.org/CorpusID:10171391>.
 11. Janet, J. A., Luo, R. C., & Kay, M. G. (1995). "The essential visibility graph: An approach to global motion planning for autonomous mobile robots." In *Proceedings of the 1995 IEEE International Conference on Robotics and Automation*, Vol. 2, 1958–1963. doi: 10.1109/ROBOT.1995.526023.
 12. Dong, H., et al. (2010). "The Path Planning for Mobile Robot Based on Voronoi Diagram." In *2010 Third International Conference on Intelligent Networks and Intelligent Systems*, pp. 446–449. doi: 10.1109/ICINIS.2010.105.
 13. Hart, P., Nilsson, N., & Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." In *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. doi: 10.1109/tssc.1968.300136. Available at: <https://doi.org/10.1109/tssc.1968.300136>.
 14. Fox, D., Burgard, W., & Thrun, S. (1997). "The dynamic window approach to collision avoidance." In *IEEE Robotics Automation Magazine*, 4(1), 23–33. doi: 10.1109/100.580977.
 15. Roesmann, C., et al. (2012). "Trajectory modification considering dynamic constraints of autonomous robots." In *ROBOTIK 2012; 7th German Conference on Robotics*, pp. 1–6.
 16. Edmonds, J. M., & Johnson, D. S. (1973). "The Chinese postman problem." *Mathematical Programming*, 4(1), 113–126.

LIST OF TABLES

17. Edmonds, J. M. (1965). "Paths, trees, and flowers." In *Proceedings of the 6th Annual Symposium on Foundations of Computer Science*, pp. 1-17.
18. Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). "Exploring network structure, dynamics, and function using NetworkX." In *Proceedings of the 7th Python in Science Conference (SciPy2008)*.
19. Hagberg, A. A., & Schult, D. A. (2008). "NetworkX: Network Analysis in Python." Available at: <https://networkx.org/documentation/stable/>.
20. Wikipedia contributors. "Route inspection problem." *Wikipedia, The Free Encyclopedia*. Available online at: https://en.wikipedia.org/wiki/Route_inspection_problem.
21. Fox, D., Garcia, A., Gerkey, B. P., Harrison, B., Kirby, M., Mitchell, J., ... Woods, D. (2014). "The design and implementation of a ROS 2 client library." arXiv preprint arXiv:1404.4734.
22. Martínez, J. M., & Durillo, J. J. (2008). "Task allocation for multi-agent systems: A review." *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(4), 665-681.
23. Lovász, L. (1973). "Matchings and covers in graphs." *Combinatorica*, 3(3), 283-300.
24. ROS2 Executors Documentation. *ROS2 Executors Overview*. Available online at: <https://design.ros2.org/articles/executors.html>.
25. Fox, D. (2001). "KLD-Sampling: Adaptive Particle Filters." In *Neural Information Processing Systems*. Available at: <https://api.semanticscholar.org/CorpusID:10171391>.
26. Meigu, G. (1960). "On the Shortest Circuit of a Graph." *Journal of Mathematics*, 3(4), 257-264.

LIST OF TABLES

27. Papadimitriou, C. H., & Steiglitz, K. (1982). "Combinatorial Optimization: Algorithms and Complexity." *Dover Publications*.
28. Thrun, S., Burgard, W., & Fox, D. (2005). "Probabilistic Robotics." *MIT Press*.
29. Pearl, J. (1984). "Heuristics: Intelligent Search Strategies for Computer Problem Solving." *Addison-Wesley*.
30. Golub, G. H., & Van Loan, C. F. (1996). "Matrix Computations." *Johns Hopkins University Press*.
31. Latombe, J. C. (1991). "Robot Motion Planning." *Kluwer Academic Publishers*.
32. Arkin, R. C. (1998). "Behavior-Based Robotics." *MIT Press*.
33. Lozano-Pérez, T. (1983). "Spatial Planning: A Configuration Space Approach." *IEEE Transactions on Computers*, 32(2), 108–120. doi: 10.1109/TC.1983.1676196.
34. Ayanian, N., & Kumar, V. (2010). "Decentralized Feedback Controllers for Multiagent Teams in Environments with Obstacles." *IEEE Transactions on Robotics*, 26(5), 878–887. doi: 10.1109/TRO.2010.2062570.
35. Khatib, O. (1986). "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots." *International Journal of Robotics Research*, 5(1), 90–98. doi: 10.1177/027836498600500106.
36. Sutton, R. S., & Barto, A. G. (2018). "Reinforcement Learning: An Introduction." *MIT Press*.
37. Carpin, S., & Pagello, E. (2002). "On Parallel RRTs for Multi-Robot Systems." *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Vol. 4, 605–611.

LIST OF TABLES

- 38. Gini, M., & Rosenschein, J. S. (1992). "Multiagent Planning and Coordination." *Artificial Intelligence Review*, 6(1), 1–7.
- 39. Kuipers, B., & Byun, Y. T. (1991). "A Robot Exploration and Mapping Strategy Based on a Semantic Hierarchy of Spatial Representations." *Journal of Robotics Research*, 9(1), 47–70.
- 40. Barto, A. G., Sutton, R. S., & Watkins, C. J. C. H. (1995). "Learning and Sequential Decision Making." *Proceedings of the IEEE*, 83(5), 957–971.
- 41. Dijkstra, E. W. (1959). "A Note on Two Problems in Connexion with Graphs." In *Numerische Mathematik*, 1, 269–271.
- 42. Mathew, N., Smith, S. L., & Waslander, S. L. (2013). "A Graph-Based Approach to Multi-Robot Rendezvous for Recharging in Persistent Tasks." *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 3497–3502. doi: 10.1109/ICRA.2013.6631111.
- 43. Bronstein, I. N., & Semendjajew, K. A. (1985). "Handbook of Mathematics." *Springer-Verlag*.
- 44. Sedgewick, R., & Wayne, K. (2011). "Algorithms (4th Edition)." *Addison-Wesley*.

Acknowledgements

Rivolgo innanzitutto un ringraziamento particolare ai miei genitori per l'incoraggiamento e il supporto costante: il loro sostegno ha rappresentato un punto di riferimento inestimabile, sia nei momenti di maggiore tensione sia in quelli di maggiore impegno. In modo speciale, ringrazio mia madre, colei che mi è rimasta accanto più di tutti, offrendo consigli preziosi anche quando, a volte, potevano sembrare troppo insistenti.

Desidero inoltre manifestare la mia gratitudine alla Prof.ssa Alessia Saggese, relatrice di questo lavoro di tesi, per avermi offerto l'opportunità di mettere in pratica le conoscenze acquisite nel mio percorso magistrale, in particolare nell'ambito della robotica. La sua guida e disponibilità hanno contribuito in maniera significativa al mio sviluppo professionale e accademico.

Non posso tralasciare i miei amici, vicini e lontani, la cui presenza e il cui incoraggiamento hanno alleggerito la pressione dei periodi più intensi di studio. Nonostante la distanza e i diversi impegni di ciascuno, il loro sostegno e la loro vicinanza, seppur non vissuti come un tempo, si sono rivelati preziosi e mi hanno permesso di affrontare con maggiore serenità le sfide di questo cammino.

Concludo rivolgendo un pensiero anche a me stesso, per la determinazione e la costanza dimostrate nell'affrontare le sfide incontrate durante questo percorso: ogni ostacolo superato è stato un'opportunità di crescita, sia sul piano personale sia su quello accademico.